Prefazione

Intorno al 1990, io e mia moglie abbiamo discusso se dovessimo trasferirci o meno. Stavamo diventando irrequieti dopo aver vissuto a lungo nella stessa casa. Per mancanza di spazio, il mio studio era stato trasformato in una cameretta per bambini. Avevo decisamente bisogno di una stanza tutta mia per finire la prima edizione di questo libro. Mia moglie pensava che la sua cucina fosse troppo piccola.

Si è presentata l'opportunità di acquistare un lotto edificabile in un nuovo piano di sviluppo. Abbiamo raccolto informazioni, cercato un architetto, sottoscritto il piano. Tuttavia, non eravamo sicuri di voler davvero lasciare la nostra casa. Era situato molto bene in una strada senza uscita, con un giardino esposto a sud e un parco giochi di fronte. Quando è stato chiesto, i nostri bambini ci hanno detto che non volevano trasferirsi in un nuovo quartiere.

Così abbiamo chiesto ad un architetto la possibilità di ricostruire la nostra casa. Ha prodotto un progetto in cui la casa modificata aveva una cucina più grande e quattro stanze extra. Ci hanno venduto l'idea immediatamente. Ci siamo detti che questa ricostruzione sarebbe stata anche più economica, anche se l'architetto non poteva ancora darci una stima affidabile dei costi.

Dopo aver riflettuto ancora sul piano di ricostruzione, abbiamo deciso che questa era la strada da percorrere. Mio fratello, che lavora nel settore edile, ci ha avvertito del disordine che creerebbe. Abbiamo pensato di potercela fare. Abbiamo avviato la procedura per ottenere il permesso, che richiede almeno sei mesi nei Paesi Bassi – e costa denaro (questo non è stato contabilizzato).

Ad agosto, un anno dopo, eravamo finalmente pronti per iniziare. La ricostruzione è stata stimata in 60 giorni lavorativi al massimo. Sfortunatamente, il contratto non prevedeva alcuna multa in caso di superamento di questo periodo. Abbiamo concordato un prezzo fisso. Alcune cose, come il nuovo cablaggio elettrico, una nuova unità di riscaldamento centrale e il costo dell'impianto idraulico non erano inclusi. Non sapevamo a malapena quanto sarebbero costati quegli "extra" alla fine. Li abbiamo valutati sul retro di una busta e ci siamo sentiti sicuri.

Il 15 settembre è stato piantato il primo mucchio. Contando sul bel tempo per tutto l'autunno, questo avrebbe dovuto significare che tutto sarebbe finito entro Natale. L'impresario edile, tuttavia, aveva altri impegni urgenti e all'inizio i progressi furono piuttosto lenti. Durante il primo mese è stata eseguita circa una settimana di lavoro.

In ottobre è stato necessario rimuovere parte del tetto. Potremmo interrompere il lavoro fino alla primavera successiva (l'opzione sicura) o continuare, piuttosto complicato in un paese umido come il nostro. Abbiamo pregato per alcune settimane asciutte e abbiamo deciso di andare avanti. L'appaltatore ha iniziato a demolire parte della nostra casa. Nel farlo, sono emerse alcune sorprese e una parte ancora più grande della casa ha dovuto essere demolita.

Siamo stati davvero fortunati: ha piovuto solo per due giorni mentre il nostro tetto era aperto. Le nostre camere da letto sono diventate piuttosto bagnate e la cucina è stata allagata. A novembre, il nuovo tetto era aperto e potevamo dormire di nuovo tranquillamente.

Entro la fine di novembre, stavamo diventando nervosi. C'era ancora molto lavoro da fare ma più volte gli operai non si sono fatti vedere. Nel frattempo, avevamo preso accordi per l'installazione della nostra nuova cucina la settimana prima di Natale. Prima di questo, una porta doveva essere tagliata in un muro di mattoni esistente. Il vecchio impianto di riscaldamento centrale è stato posizionato proprio dietro quel muro e doveva essere prima rimosso. Il nuovo impianto di riscaldamento, purtroppo, non era ancora disponibile (l'autunno è l'alta stagione per gli impianti di riscaldamento).

I lavori sono proseguiti per quanto possibile. Fu eretto un nuovo muro, dopo il quale potemmo entrare nella nostra (vecchia) cucina solo dall'esterno. Per un po' abbiamo anche vissuto senza cucina. Per farla breve, l'appaltatore ce l'ha fatta, ma solo a malapena. La nuova cucina è stata installata. Al piano di sopra, invece, restava molto lavoro. Il progetto è stato finalmente completato entro la fine di gennaio, con sole sei settimane di ritardo.

Durante la ricostruzione, la vita era stata piuttosto provvisoria. Il mio computer era riposto in soffitta. I bambini non avevano praticamente spazio per giocare al chiuso. La polvere era ovunque. Queste circostanze possono essere tollerate per un po', ma verso la fine siamo diventati frenetici. Anche se i lavori sembravano terminati entro la fine di gennaio, molto restava ancora da fare: le stanze dovevano essere tinteggiate e decorate, e tutto ciò che era stato imballato doveva essere disimballato di nuovo. Ci vollero molti altri mesi prima che la vita riprendesse il suo corso normale.

Alcuni mesi dopo, alcune delle nuove assi di legno sulla facciata posteriore hanno iniziato a rompersi. Si erano espansi durante la calura estiva; o la lingua era troppo larga o il solco troppo stretto. Questo, e vari altri problemi minori furono, alla fine, risolti.

Sul lato finanziario: varie piccole spese non contabilizzate sommate a una bella somma. Non sono ancora sicuro se abbiamo scelto l'opzione più economica, ma sono assolutamente sicuro che abbattere una casa e ricostruirla mentre stai ancora cercando di viverci sia un incubo. In questo senso, mio fratello aveva più che ragione.

Dopo che il progetto di ricostruzione della casa e il lavoro per la prima edizione di questo libro sono stati completati, ho rivolto la mia attenzione al riordino del nostro giardino. Ho progettato un giardino con vari bordi, terrazze, una casa estiva e uno stagno. E ho eseguito tutto il lavoro. Ho commesso un grosso errore su questo secondo progetto, che si è manifestato solo un paio d'anni dopo quando ho voluto ridipingere la facciata posteriore. Per fare ciò, ho dovuto mettere le gambe della scala nello stagno. Quindi ho fatto qualche rielaborazione e spostato lo stagno.

Questa storia è abbastanza tipica di un progetto di sviluppo software. Anche il software spesso viene consegnato in ritardo, non soddisfa le sue specifiche ed è, inoltre, difettoso. I progetti software tendono anche a sottovalutare l'impatto di fattori non tecnici. La crescente consapevolezza di ciò alla fine degli anni '60 ha dato origine all'espressione "crisi del software". Fortunatamente, abbiamo fatto alcuni progressi da quando il termine "ingegneria del software" è stato coniato per la prima volta nel 1968.

INGEGNERIA SOFTWARE

Il campo dell'ingegneria del software mira a trovare risposte ai numerosi problemi che i progetti di sviluppo software possono incontrare quando si costruiscono grandi sistemi software. Tali sistemi sono complessi a causa delle loro dimensioni, perché sono sviluppati da un team che coinvolge persone di diverse discipline e perché verranno modificati regolarmente per soddisfare le mutevoli esigenze, sia durante lo sviluppo che dopo l'installazione.

L'ingegneria del software è ancora un campo giovane rispetto ad altre discipline ingegneristiche. Tutte le discipline hanno i loro problemi, in particolare quando i progetti vanno oltre le competenze degli ingegneri. Sembra che i progetti di sviluppo software estendano continuamente l'esperienza dei loro ingegneri.

L'argomento è in rapido movimento e ci sono più domande che risposte. Tuttavia, una serie di principi e pratiche si sono evoluti nei 40 anni di esistenza del campo. Per promuovere e mantenere l'ingegneria del software come disciplina professionale, la IEEE Computer Society e ACM hanno realizzato un progetto congiunto - SWEBOK - per identificare e convalidare il corpo delle conoscenze di ingegneria del software. Questo progetto è durato dal 1998 al 2001.

Questo libro affronta tutte le aree di conoscenza identificate nel progetto SWEBOK. [1] Naturalmente, l'attenzione relativa prestata ai singoli argomenti e al modo in cui questi argomenti vengono trattati riflette la mia visione del campo. Questa visione può essere riassunta come segue:

- Ciò che è teoria oggi può diventare pratica domani. Per questo motivo, non mi sono limitato a una discussione di pratiche consolidate. Piuttosto, presto anche attenzione a metodi e tecniche promettenti che non hanno ancora superato l'ambiente di ricerca o lo hanno appena fatto: valutazione quantitativa della qualità del software, ingegneria del software basata sui componenti e ingegneria del software orientata ai servizi, solo per citarne alcuni.
- Possiamo imparare molto dalla nostra storia. Non parlo solo di tecniche che hanno dimostrato il loro valore e che sono ampiamente utilizzate oggi. Discuto anche di sviluppi che sono ormai considerati vicoli ciechi. Sapere *perché* una certa tecnica non viene più utilizzata è spesso prezioso. La mia discussione sui modelli di stima dei costi nel **Capitolo 7** è un esempio calzante.

- Tutto cambia. I requisiti cambiano mentre lo sviluppo è ancora in corso. Le persone entrano ed escono dal team di progetto. La funzionalità di un set di strumenti cambia prima che i sistemi sviluppati con esso vengano sostituiti. E così via. Il cambiamento è un tema ricorrente in questo libro.
- Gli aspetti umani e sociali sono centrali. La maggior parte dei capitoli di questo libro portano titoli che suonano abbastanza tecnici. All'interno di questi stessi capitoli, tuttavia, tocco regolarmente gli aspetti umani e sociali del mestiere. Ad esempio, l'elicitazione dei requisiti non è affatto una questione puramente tecnica e la progettazione di un sistema è fortemente influenzata dalle precedenti esperienze, sia positive che negative, del progettista.
- L'ingegneria del software sta diventando sempre più eterogenea, come dimostrano i recenti sviluppi (Vliet, **2006**):
    - – Sviluppo software distribuito, coinvolgendo team di culture diverse e sparsi in tutto il mondo.
    - – La combinazione di software sviluppato internamente con COTS, open source e altri software acquisiti esternamente.
    - – La combinazione di approcci di sviluppo tradizionali basati sui documenti con approcci di sviluppo agile più recenti e guidati dalle persone.

Come conseguenza di questo cambiamento, il controllo dello sviluppo del software è diventato molto più difficile.

Le persone attivamente coinvolte nello sviluppo e nella manutenzione del software - programmatori, analisti, project manager - e studenti di informatica e ingegneria del software devono essere consapevoli dei problemi incontrati dallo sviluppo di software su larga scala e delle soluzioni proposte.

Credo fermamente che nessuna delle soluzioni proposte sia un proiettile d'argento: CASE, sviluppo software orientato agli oggetti, riutilizzo del software, progettazione architettonica, specifiche formali, modelli di processo, ingegneria del software basata su componenti, ingegneria del software orientata ai servizi; ognuno contribuisce con il proprio acaro. I problemi fondamentali, tuttavia, rimarranno. I sistemi software sono artefatti estremamente complessi. La loro riuscita realizzazione richiede esperienza e talento da parte dei loro designer. Se applicati in modo ponderato e coscienzioso, i metodi e le tecniche discussi in questo libro possono aiutarti a diventare un ingegnere del software professionista.

APPRENDIMENTO SULL'INGEGNERIA DEL SOFTWARE

La maggior parte dei capitoli di questo libro possono essere letti e studiati indipendentemente. In un ambiente scolastico, l'istruttore ha un ampio grado di libertà nella scelta degli argomenti di questo libro e nell'ordine in cui trattarli. Si raccomanda che un primo corso di ingegneria del software affronti almeno gli argomenti discussi nei **capitoli 1** – **3** e **9** – **14** (in quest'ordine). Materiale aggiuntivo può essere scelto a piacimento dagli altri capitoli o

essere utilizzato come materiale per un corso secondario. Due ovvi gruppi di materiale per un corso secondario sono i capitoli 4 – 8 e 16 – 20 (vedi Figura 1).

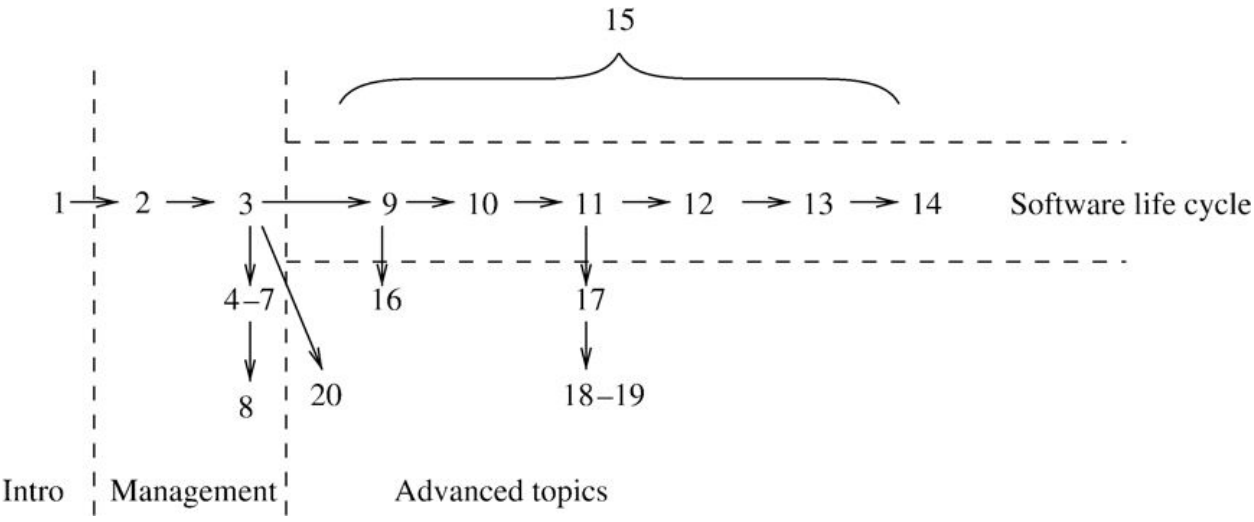


Figura 0.1Come è organizzato il libro

Un problema ricorrente nell'insegnamento dell'ingegneria del software è quando e come affrontare i problemi di gestione dei progetti. Gli studenti di informatica hanno spesso difficoltà ad apprezzare l'importanza di questioni come l'organizzazione del team e la stima dei costi. I professionisti del software sanno dalle trincee che questi problemi non tecnici sono importanti almeno quanto quelli tecnici. È più probabile che gli studenti di informatica o ingegneria del software comprendano l'importanza delle questioni di gestione verso la fine del corso, possibilmente dopo che sono stati coinvolti in una sorta di lavoro pratico. Tuttavia, una breve trattazione delle questioni sollevate nei capitoli 2 e 3 dovrebbe essere fornita all'inizio del corso.

Molto di ciò che viene detto in questo libro sembra ovvio. In effetti, lo è. Come ha detto un relatore a una conferenza sull'educazione all'ingegneria del software: "Non puoi insegnarlo, puoi solo predicarlo". Quindi questo libro è un lungo sermone su come esercitarsi nello sviluppo di software. Proprio come non puoi diventare un bravo artigiano di falegnameria leggendo un libro di testo sull'argomento, non puoi diventare un serio ingegnere del software semplicemente leggendo e assorbendo il materiale contenuto in questo libro. Devi esercitarti anche tu.

Fare un lavoro pratico in un ambiente universitario non è facile. I molti rischi che corrono i progetti di sviluppo software nella vita reale non possono essere realisticamente imitati in un progetto a termine. Tuttavia, alcuni problemi ricorrenti nello sviluppo del software possono essere affrontati con successo. Ad esempio, a piccoli gruppi di studenti può essere chiesto di progettare, implementare e testare un sistema non banale, che un altro gruppo deve mantenere.

figura 2 mostra come gli scolari di Amsterdam hanno imparato a nuotare intorno al 1900. Mio padre è cresciuto in campagna e ha imparato a proprie spese. Suo padre gli ha semplicemente legato una corda intorno alla vita, lo ha gettato nel fiume che scorreva davanti alla casa e ha gridato: "Nuota". Al giorno d'oggi, gli scolari olandesi, in generale, ottengono il loro primo certificato di nuoto prima di entrare a scuola. Le loro lezioni di nuoto iniziano in modo molto dolce, in una piscina per bambini, accanto alla mamma e con tanto materiale per tenerli a galla. A poco a poco, la quantità di materiale galleggiante si riduce e la piscina diventa più profonda. Non si spaventano e di solito si godono la lezione di nuoto.

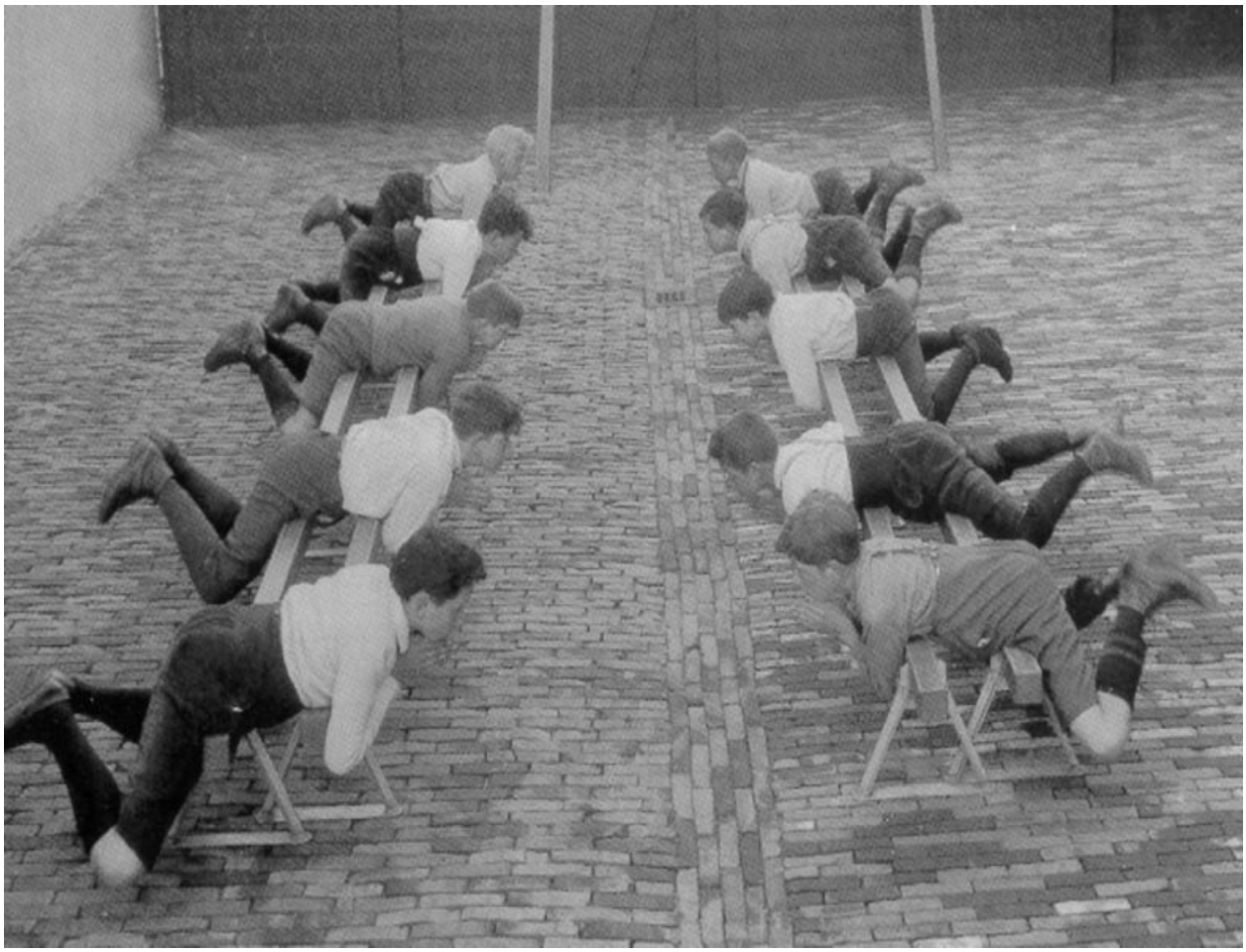


Figura 0.2L'equivalente di nuoto di un corso per corrispondenza in ingegneria del software © The Municipal Archives of Amsterdam. Riprodotto con permesso

Una gamma simile di possibilità è possibile in un corso di ingegneria del software. L'equivalente del nuoto in terraferma non è da raccomandare. Anche farlo nel modo più duro, coinvolgendo lo studente in un progetto reale, ha i suoi problemi. Lo studente può, in senso figurato, annegare nelle complessità pratiche quotidiane del progetto. Sembra più appropriato una sorta di schema intermedio che coinvolga aspetti della "vita reale" in un ambiente protetto, o una sequenza di esperienze educative con una quantità crescente di realismo. I problemi della formazione in ingegneria del software sono affrontati nella

conferenza annuale sulla formazione in ingegneria del software. Vedi anche (Inverardi e Jazayeri, **2006**) per una panoramica sullo stato dell'arte.

Oltre a questo lavoro pratico, gli esercizi alla fine di ogni capitolo forniscono ulteriore materiale di apprendimento. Gli esercizi semplicemente numerati pongono domande relativamente semplici sul capitolo appena letto. Gli esercizi contrassegnati con una ♡ o ♠ richiedono al lettore di riflettere seriamente su questioni importanti o di studiare fonti aggiuntive per approfondire la sua comprensione. [2] Gli esercizi contrassegnati da a ♡ possono richiedere un'ora per rispondere. Gli esercizi contrassegnati ♠ possono richiedere più di un giorno. I semplici esercizi danno solo una conoscenza superficiale del campo. Una profonda conoscenza dell'ingegneria del software sarà sviluppata solo se ti sarai fatto le ossa in alcuni degli esercizi contrassegnati. Questi esercizi ti costringono a pensare e riflettere sul materiale discusso. La parte di riflessione è importante quanto la semplice risposta. Le risposte a questi esercizi e altro materiale didattico possono essere ottenuti da **www.cs.vu.nl/~hans/SEbook.html** .

COSA E' CAMBIATO?

L'ingegneria del software è un campo in rapida evoluzione. La preparazione di questa terza edizione richiedeva quindi modifiche in ogni capitolo. Ma alcuni capitoli sono cambiati più di altri. Le modifiche principali sono le seguenti:

- Ho aggiunto materiale sui metodi di sviluppo agile, in particolare nei **capitoli 1** – **8** .
- Ho aggiunto tre capitoli completamente nuovi, sull'ingegneria del software basata sui componenti, sull'orientamento ai servizi e sullo sviluppo del software globale.
- Ho notevolmente aggiornato i capitoli sull'ingegneria dei requisiti (9) e sull'architettura del software (11).
- Ho raccolto il materiale sulle questioni di notazione in un **capitolo (10)** .
- Ho integrato il materiale rimanente sull'analisi e la progettazione orientata agli oggetti nel capitolo sulla progettazione del software (12).
- Ho integrato il materiale sull'affidabilità del software nel capitolo sul test del software (13).
- Ho abbandonato il capitolo sulle specifiche formali.

RINGRAZIAMENTI

Il presente testo è in realtà una quinta edizione. Le prime due edizioni sono apparse solo in olandese. Ho utilizzato questo materiale molte volte nei corsi, sia per studenti universitari che per professionisti del software. Queste persone, consciamente o inconsciamente, hanno contribuito a plasmare il testo così com'è. Ho ricevuto molti suggerimenti utili da Remco de Boer, Rik Farenhorst, Steven Klusener, Philippe Kruchten (University of British Columbia), Patricia Lago, Judith Stafford (Tufts University) e Witold Suryn (University of Quebec). Un

ringraziamento speciale va a Gerrit van der Veer, coautore del capitolo 16 , e Michel Chaudron e Ivica Crnkovic, coautore del capitolo 18 .

Shena Deuchars di Mitcham Editorial Services ha svolto un ottimo lavoro come copy-editor. Molte persone di John Wiley & Sons hanno contribuito a questo libro. Emma Cooper si è occupata di tutto il lavoro di produzione. Claire Jardine si è occupata di una serie di faccende domestiche. Grazie anche al Commissioning Editor, Jonathan Shipley.

La litografia in copertina si chiama 'WaterfalL'(MC Escher, 1961). È appropriato sia nel nome che nel messaggio.

Infine, ringrazio Marjan, Jasper e Marieke per la loro pazienza e supporto. Il superamento del programma di questo progetto è stato peggiore di quello di molti progetti di sviluppo software.

Hans van Vliet

Amsterdam, ottobre 2007

[1] La versione 2004 della Guida SWEBOK elenca le seguenti aree di conoscenza: requisiti del software, progettazione del software, costruzione del software, test del software, manutenzione del software, gestione della configurazione del software, gestione dell'ingegneria del software, processo di ingegneria del software, strumenti e metodi di ingegneria del software e qualità del software. L'area di costruzione del software copre sia la codifica che i test di unità e integrazione; di questi, in questo libro vengono trattati solo i test unitari e di integrazione. Per ulteriori informazioni sul progetto SWEBOK, vedere www.sWebok.org .

[2] Invece di scrivere sempre "lui o lei", userò pronomi maschili in tutto questo testo per brevità.

1 Introduzione

OBIETTIVI FORMATIVI

- Comprendere la nozione di **ingegneria del software** e perché è importante
- Apprezzare gli aspetti tecnici (ingegneristici), gestionali e psicologici dell'ingegneria del software
- Comprendere le somiglianze e le differenze tra l'ingegneria del software e altre discipline ingegneristiche
- Conoscere le fasi principali di un progetto di sviluppo software
- Apprezzare le dimensioni etiche nell'ingegneria del software
- Essere consapevoli del lasso di tempo e della misura in cui i nuovi sviluppi influiscono sulla pratica dell'ingegneria del software

L'ingegneria del software riguarda metodi e tecniche per sviluppare grandi sistemi software. La metafora ingegneristica viene utilizzata per enfatizzare un approccio sistematico allo sviluppo di sistemi che soddisfino i requisiti ei vincoli organizzativi. Questo capitolo fornisce una breve panoramica del campo e indica le tendenze emergenti che influenzano il modo in cui viene sviluppato il software.

L'informatica è ancora un campo giovane. I primi computer furono costruiti a metà degli anni '40, da quando il campo si è sviluppato enormemente.

Le applicazioni dei primi anni dell'informatizzazione possono essere così caratterizzate: i programmi erano piuttosto piccoli, sicuramente rispetto a quelli attualmente in costruzione; erano scritti da una persona; erano scritti e utilizzati da esperti nell'area di applicazione interessata. I problemi da risolvere erano per lo più di natura tecnica e l'enfasi era sull'espressione efficiente di algoritmi noti in alcuni linguaggi di programmazione. L'input consisteva tipicamente in dati numerici, letti da supporti come nastri perforati o schede perforate. L'output, anche numerico, era stampato su carta. I programmi erano eseguiti offline. Se il programma avesse contenuto errori, il programmatore avrebbe studiato un dump di memoria ottale o esadecimale. A volte, l'esecuzione del programma sarebbe seguita dalla lettura binaria dei registri macchina sulla console.

A quei tempi difficilmente esistevano società di sviluppo software indipendenti. Il software era per lo più sviluppato da fornitori di hardware e distribuito gratuitamente. Questi fornitori a volte creano gruppi di utenti per discutere i requisiti, che hanno incorporato nel loro software. Questo supporto allo sviluppo del software è stato visto come un servizio per i loro clienti.

Le applicazioni odierne sono piuttosto diverse sotto molti aspetti. I programmi odierni sono spesso molto ampi e sono sviluppati da team che collaborano per periodi che abbracciano diversi anni. Queste squadre possono essere sparse in tutto il mondo. I programmatori non sono i futuri utenti del sistema che sviluppano e non hanno una conoscenza approfondita

dell'area applicativa in questione. I problemi che vengono affrontati riguardano sempre di più la vita di tutti i giorni: cassieri automatici, prenotazione aerea, amministrazione salariale, commercio elettronico, sistemi automobilistici, ecc. Mettere un uomo sulla luna non era concepibile senza i computer.

Negli anni '60, le persone iniziarono a rendersi conto che le tecniche di programmazione erano rimaste indietro rispetto agli sviluppi del software sia in termini di dimensioni che di complessità. Per molte persone, la programmazione era ancora *un'arte* e non era mai diventata un *mestiere*. Un ulteriore problema era che molti programmatori non erano stati formati formalmente sul campo. Avevano imparato facendo. Dal punto di vista organizzativo, le soluzioni ai problemi spesso comportavano l'aggiunta di un numero sempre maggiore di programmatori al progetto, il cosiddetto approccio "a milioni di scimmie".

Di conseguenza, il software veniva spesso consegnato troppo tardi, i programmi non si comportavano come previsto dall'utente, raramente i programmi erano adattabili alle mutate circostanze e molti errori venivano rilevati solo dopo che il software era stato consegnato al cliente. Questa divenne nota come la "crisi del software".

Questo tipo di problema si è davvero manifestato negli anni '60. Sotto gli auspici della NATO, due conferenze furono dedicate all'argomento nel 1968 e nel 1969 (Naur e Randell, **1968**; Buxton e Randell, **1969**). Qui il termine "ingegneria del software" è stato coniato in un senso alquanto provocatorio. Non dovrebbe essere possibile costruire software come si costruiscono ponti e case, partendo da basi teoriche e utilizzando tecniche progettuali e costruttive solide e collaudate, come in altri campi dell'ingegneria?

Il software ha uno scopo organizzativo. Le ragioni per intraprendere un progetto di sviluppo software variano. A volte, una soluzione a un problema non è fattibile senza l'ausilio di computer, come le previsioni meteorologiche o le rilevazioni bancarie automatizzate. A volte, il software può essere utilizzato come veicolo per nuove tecnologie, come la composizione, la produzione di chip o viaggi spaziali con equipaggio. In altri casi ancora, il software può aumentare il servizio all'utente (automazione della libreria, e-commerce) o risparmiare denaro (controllo automatizzato delle scorte).

In molti casi, il guadagno economico previsto sarà una forza trainante importante per l'automazione. Tuttavia, potrebbe non essere sempre facile dimostrare che l'automazione fa risparmiare denaro (basti pensare all'automazione degli uffici) perché, oltre al risparmio diretto sui costi, il guadagno economico può manifestarsi anche in cose come una produzione più flessibile o un utente più veloce o migliore servizio. In entrambi i casi, lo sviluppo del software è un'attività che crea valore.

Boehm (**1981**) stimava che la spesa totale per il software negli Stati Uniti fosse di 40 miliardi di dollari nel 1980. Si trattava di circa il 2% del PIL. Nel 1985, la spesa totale era salita a 70 miliardi di dollari negli Stati Uniti ea 140 miliardi di dollari nel mondo. Boehm e Sullivan (**1999**)

stimarono che la spesa annuale per lo sviluppo del software nel 1998 fosse di 300-400 miliardi di dollari negli Stati Uniti e il doppio di tale importo nel mondo.

Nel 2024, la spesa globale per il software ha raggiunto i 675 miliardi di dollari, con un aumento nominale di quasi il 50% rispetto ai 454 miliardi di dollari del 2020.

Se vuoi saperne di più, puoi guardare qui: https://www.wipo.int/en/web/global-innovation-index/w/blogs/2025/global-software-spending

Questi i dati rispetto il PIL (GDP in inglese)

Table 1: Top Economies for Software Spending, in USD Billion (left) and Scaled as a Percentage of GDP (right), 2024

Top 10 Economies	2024 Software Spending in USD billion	Unscaled Rank	Top 10 Economies	2024 Software Spending as a Percentage of GDP	Scaled Rank
United States	368.52	1	United States	1.26	1
China	61.77	2	Spain	0.71	2
Germany	22.23	3	Ireland	0.68	3
United Kingdom	20.32	4	Sweden	0.68	4
France	19.28	5	Iran	0.66	5
Italy	14.24	6	Switzerland	0.65	6
Canada	14.03	7	Canada	0.63	7
Japan	13.16	8	Denmark	0.62	8
Spain	12.26	9	France	0.61	9
India	11.41	10	Portugal	0.61	10

Quindi il *costo* del software è di fondamentale importanza. Ciò riguarda non solo il costo di sviluppo del software, ma anche il costo di mantenere operativo il software una volta consegnato al cliente. Nel corso del tempo, i costi hardware sono diminuiti drasticamente. I costi dell'hardware ora rappresentano generalmente meno del 20% della spesa totale (vedere la **figura 1.1**). Il restante 80% è composto da tutti i costi non hardware: il costo di programmatori, analisti, gestione, formazione utenti, assistenza di segreteria, ecc.

Un aspetto strettamente legato al costo è *la produttività.* Negli anni '80, la richiesta di personale addetto all'elaborazione dati è aumentata del 12% all'anno, mentre la popolazione di persone che lavorano nell'elaborazione dati e la produttività di queste persone sono cresciute ciascuna di circa il 4% all'anno (Boehm, **1987a**). Questa situazione non è sostanzialmente cambiata (Jones, **1999**). L'effetto netto è un divario crescente tra domanda e offerta. Il risultato è sia un arretrato rispetto alla manutenzione del software esistente sia un rallentamento nello sviluppo di nuove applicazioni. L'effetto combinato può avere ripercussioni sul vantaggio competitivo di un'organizzazione, soprattutto in presenza di forti vincoli di time-to-market. Questi sviluppi hanno portato a un passaggio dalla *produzione* di software *all'utilizzo del* software. Torneremo su questo argomento nella **Sezione 1.6** e nei **Capitoli 17** – **19** .

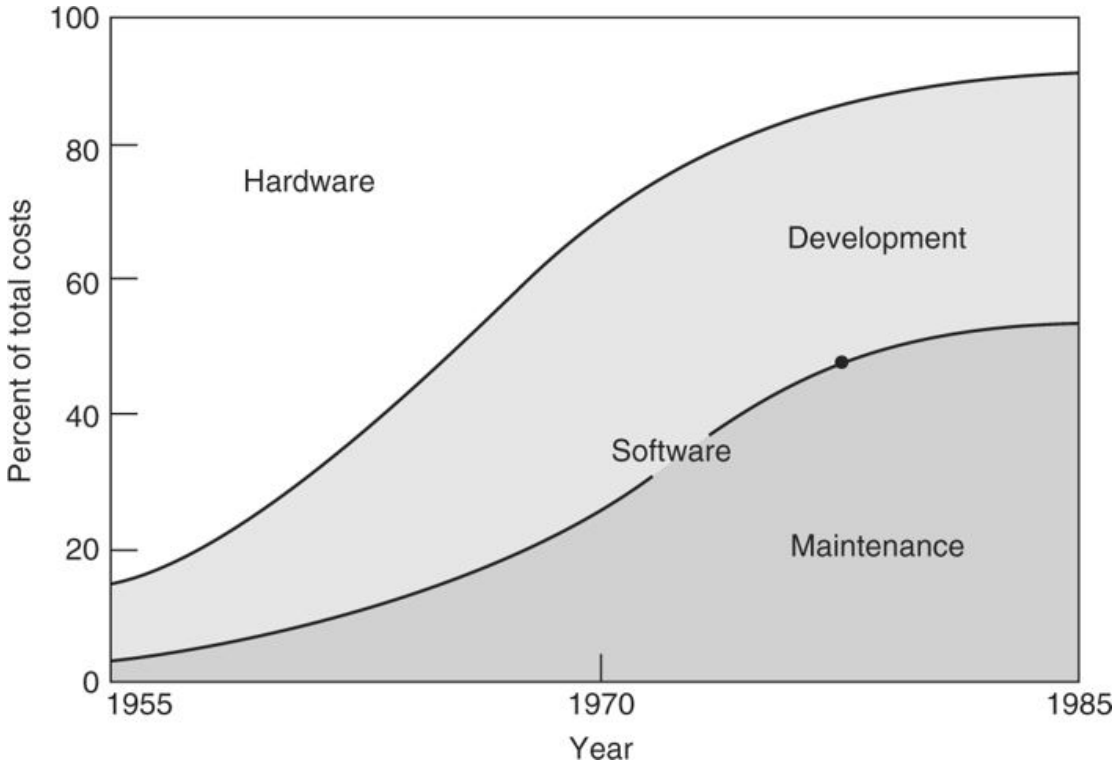


Figura 1.1 Distribuzione relativa dei costi hardware e software (Fonte: BW Boehm, Software Engineering, IEEE Transactions on Computers, © 1976 IEEE. Riprodotto con autorizzazione.)

Le questioni relative ai costi e alla produttività dello sviluppo del software meritano la nostra seria attenzione. Tuttavia, questa non è la storia completa. La società è sempre più dipendente dal software. La qualità dei sistemi che sviluppiamo determina sempre più la qualità della nostra esistenza. Si consideri come esempio il seguente messaggio di un quotidiano olandese del 6 giugno 1980, sotto il titolo "Gli americani hanno visto arrivare i russi":

- Martedì scorso, per un breve periodo, gli Stati Uniti hanno portato i loro bombardieri atomici e missili nucleari in uno stato di maggiore allarme quando, a causa di un errore

del computer, un falso allarme ha indicato che l'Unione Sovietica aveva iniziato un attacco missilistico.

Gli sforzi per riparare l'errore furono apparentemente vani, poiché il 9 giugno 1980 lo stesso giornale riportava:

- Per la seconda volta in pochi giorni, un computer squilibrato riferì che l'Unione Sovietica aveva avviato un attacco nucleare contro gli Stati Uniti. Sabato scorso il DoD ha affermato il falso messaggio, che ha portato all'avviamento dei motori degli aerei dell'Aeronautica Militare strategica.

Non è sempre il mondo ad essere in pericolo. Su scala minore, gli errori nel software possono avere conseguenze molto spiacevoli, come errori di transazione nel traffico bancario; solleciti per pagare una fattura di $ 0,00; un sistema di controllo delle scorte che emette ordini troppo tardi e quindi licenzia le divisioni complete di una fabbrica.

Quest'ultimo esempio indica che gli errori in un sistema software possono avere gravi conseguenze finanziarie per l'organizzazione che lo utilizza. Un esempio di tale perdita finanziaria è la grande compagnia aerea statunitense che ha perso 50 milioni di dollari a causa di un errore nel sistema di prenotazione dei posti a sedere. Il sistema ha riportato erroneamente che i posti economici erano esauriti mentre, in realtà, ce n'erano molti disponibili. Il problema è stato rilevato solo dopo che i risultati trimestrali sono rimasti notevolmente indietro rispetto a quelli dei rispettivi periodi precedenti ea quelli dei concorrenti.

Gli errori nei sistemi automatizzati possono anche avere effetti fatali. Un settimanale di informatica conteneva il seguente messaggio nell'aprile 1983:

- Il tribunale di Düsseldorf ha dimesso una donna (54 anni), che era sotto processo per l'omicidio della figlia. Un messaggio erroneo da un sistema informatico ha fatto sì che la compagnia di assicurazioni la informasse che era gravemente malata. Si diceva che soffrisse di una forma incurabile di sifilide. Inoltre, si diceva che avesse infettato entrambi i suoi figli. In preda al panico, ha strangolato sua figlia di 15 anni e ha cercato di uccidere se stessa e suo figlio di 13 anni. Il ragazzo è scappato e, con un po' di aiuto si è arruolato, ha impedito alla donna di morire di overdose. Il giudice ha accusato l'errore del computer e ha ritenuto la donna non responsabile delle sue azioni.

Tutto questo segna l'enorme importanza del campo dell'ingegneria del software. Metodi e tecniche migliori per lo sviluppo del software possono comportare ingenti risparmi finanziari, metodi più efficaci di sviluppo del software, sistemi che si adattano meglio alle esigenze degli utenti, sistemi software più affidabili e quindi un ambiente più affidabile in cui tali sistemi funzionano. Qualità e produttività sono due temi centrali nel campo dell'ingegneria del software.

Sul lato positivo, è imperativo sottolineare gli enormi progressi compiuti dagli anni '60. Il software è onnipresente e sono stati costruiti decine di sistemi affidabili. Questi vanno da piccole applicazioni per fogli di calcolo a sistemi di composizione, sistemi bancari, browser Web e software Space Shuttle. Le tecniche ei metodi discussi in questo libro hanno contribuito al successo di questi e di molti altri progetti di sviluppo software.

1.1 CHE COS'È L'INGEGNERIA DEL SOFTWARE?

In vari testi sull'argomento si incontra una definizione del termine ingegneria del software. Una prima definizione fu data alla prima conferenza della NATO (Naur e Randell, **1968**):

- L'ingegneria del software è l'istituzione e l'uso di solidi principi di ingegneria al fine di ottenere un software economicamente affidabile e funzionante in modo efficiente su macchine reali.
- *In inglese: Software engineering is the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines*

La definizione data nel *IEEE Standard Glossary of Software Engineering Terminology* (IEEE610, **1990**) è la seguente:

- L'ingegneria del software è l'applicazione di un approccio sistematico, disciplinato e quantificabile allo sviluppo, al funzionamento e alla manutenzione del software; cioè l'applicazione dell'ingegneria al software.
- *In inglese: Software engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.*

Altre definizioni:

- "*La disciplina che si occupa della costruzione di sistemi software così **grandi** da essere costruiti da un team o da **squadre** di ingegneri*." [Ghezzi, Jazayeri, Mandrioli]
- "*Costruzione multi-persona di software multi-versione*." [Parnas]
- "La *disciplina il cui obiettivo è la produzione di software esente da errori, consegnato nei tempi e nei limiti del budget, che soddisfi le esigenze dell'utente. Inoltre, il software deve essere facilmente modificabile quando cambiano le esigenze dell'utente*."[Schach]
- *"È qui che puoi progettare grandi cose ed essere creativo".* [Taylor]

Queste e altre definizioni del termine "ingegneria del software" usano parole piuttosto diverse. Tuttavia, le caratteristiche essenziali del campo sono sempre, esplicitamente o implicitamente, presenti:

- L'ingegneria del software riguarda lo sviluppo di **grandi programmi**.

(DeRemer e Kron, **1976**) fa una distinzione tra **programmazione in grande** e **programmazione in piccolo.** Il confine tra grande e piccolo ovviamente non è netto: un programma di 100 righe è piccolo, un programma di 50 000 righe di codice non lo è certo. La programmazione in piccolo si riferisce generalmente a programmi scritti da una persona in un periodo di tempo relativamente breve. La programmazione in generale, quindi, si riferisce a lavori composti da più persone che si estendono, diciamo, per più di sei mesi. Per esempio:

- Il software della NASA Space Shuttle contiene 40 milioni di righe di codice oggetto (questo è 30 volte il software per il progetto Saturn V degli anni '60) (Boehm, **1981**);
- Il sistema operativo IBM OS360 ha richiesto 5000 anni di lavoro di sviluppo (Brooks, **1995**).

Le tecniche e gli strumenti di programmazione tradizionali mirano principalmente a supportare la programmazione in piccolo. Ciò vale non solo per i linguaggi di programmazione, ma anche per gli strumenti (come i diagrammi di flusso) ei metodi (come la programmazione strutturata). Questi non possono essere trasferiti direttamente allo sviluppo di grandi programmi.

In effetti, il termine programma – nel senso di un pezzo di software autonomo che può essere invocato da un utente o da qualche altro componente del sistema – non è adeguato qui. I progetti di sviluppo software odierni danno luogo a sistemi contenenti un gran numero di programmi (interconnessi) o componenti.

- Il tema centrale è padroneggiare la **complessità**.

In generale, i problemi sono tali da non poter essere esaminati nella loro interezza. Si è costretti a dividere il problema in parti in modo tale che ogni singola parte possa essere colta, mentre la comunicazione tra le parti rimane semplice. La complessità totale non diminuisce in questo modo, ma diventa gestibile. In un sistema stereo ci sono componenti come un amplificatore, un ricevitore e un sintonizzatore e la comunicazione tramite un filo sottile. Nel software, ci sforziamo per una simile separazione delle preoccupazioni. In un programma per l'automazione delle biblioteche, componenti come l'interazione dell'utente, i processi di ricerca e l'archiviazione dei dati potrebbero, ad esempio, essere distinti, con strutture chiaramente indicate per lo scambio di dati tra tali componenti. Si noti che la complessità di molti software non è tanto causata dalla complessità intrinseca del problema (come nel caso degli algoritmi di ottimizzazione del compilatore o degli algoritmi numerici per la risoluzione di equazioni differenziali alle derivate parziali), quanto piuttosto dal vasto numero di dettagli che deve essere affrontato.

- Il software si **evolve**.

La maggior parte dei software modella una parte della realtà, come l'elaborazione delle richieste in una biblioteca o il monitoraggio dei trasferimenti di denaro in una banca. Questa realtà si evolve. Se il software non vuole diventare obsoleto abbastanza rapidamente, deve

evolversi con la realtà che viene modellata. Ciò significa che i costi vengono sostenuti dopo la consegna del sistema software e che dobbiamo tenere conto di questa evoluzione durante lo sviluppo.

- **L'efficienza** con cui viene sviluppato il software è di fondamentale importanza.

Il costo totale e il tempo di sviluppo dei progetti software sono elevati. Questo vale anche per la manutenzione del software. La ricerca di nuove applicazioni supera la risorsa della forza lavoro. Cresce il divario tra domanda e offerta. Le richieste di time-to-market richiedono consegne rapide. Temi importanti nel campo dell'ingegneria del software riguardano metodi e strumenti migliori e più efficienti per lo sviluppo e la manutenzione del software, in particolare metodi e strumenti che consentono l'uso e il riutilizzo dei componenti.

- La **cooperazione** regolare tra le persone è parte integrante della programmazione in generale.

Poiché i problemi sono grandi, molte persone devono lavorare contemporaneamente per risolverli. Sempre più spesso, team in diverse località geografiche lavorano insieme nello sviluppo del software. Devono esserci disposizioni chiare per la distribuzione del lavoro, metodi di comunicazione, responsabilità e così via. Tuttavia, gli accordi da soli non sono sufficienti; bisogna anche attenersi a quelle disposizioni. Al fine di applicarli, possono essere utilizzati standard o procedure. Tali procedure e standard possono spesso essere supportati da strumenti. La disciplina è una delle chiavi per il completamento con successo di un progetto di sviluppo software.

- Il software deve supportare i suoi **utenti** in modo efficace.

Il software è sviluppato per supportare gli utenti al lavoro. La funzionalità offerta dovrebbe adattarsi ai compiti degli utenti. Gli utenti che non sono soddisfatti del sistema cercheranno di aggirarlo o, nella migliore delle ipotesi, di esprimere immediatamente nuovi requisiti. Non è sufficiente costruire il sistema nel modo giusto; dobbiamo anche costruire il sistema giusto. Un supporto efficace per gli utenti significa che dobbiamo studiare attentamente gli utenti al lavoro, al fine di determinare i requisiti funzionali adeguati, e dobbiamo affrontare anche l'usabilità e altri aspetti della qualità, come l'affidabilità, la reattività e la facilità d'uso. Significa anche che lo sviluppo del software implica molto di più della semplice fornitura di software. Potrebbe essere necessario scrivere manuali per l'utente e materiale di formazione e prestare attenzione allo sviluppo dell'ambiente in cui verrà installato il nuovo sistema. Ad esempio, un nuovo sistema bibliotecario automatizzato influenzerà le procedure di lavoro all'interno della biblioteca.

- L'ingegneria del software è un campo in cui i membri di un **dominio** creano artefatti per conto dei membri di un altro dominio.

Questo aspetto è strettamente legato alle due voci precedenti. Gli ingegneri del software sono esperti in una o più aree come la programmazione in Java, l'architettura del software, i

test o il linguaggio di modellazione unificato. In genere non sono esperti nella gestione delle biblioteche, nell'avionica o nel settore bancario. Eppure, devono sviluppare sistemi per tali domini. La scarsa diffusione della conoscenza del dominio applicativo è una fonte comune di problemi nei progetti di sviluppo software.

Non solo gli ingegneri del software spesso non hanno una conoscenza fattuale del dominio per il quale sviluppano il software, ma mancano anche della sua cultura. Ad esempio, uno sviluppatore di software può scoprire l'insieme "ufficiale" di pratiche di lavoro di una determinata comunità di utenti da interviste, politiche scritte e così via; queste pratiche di lavoro vengono quindi integrate nel software. Una questione cruciale per quanto riguarda l'accettazione e il successo del sistema, tuttavia, è se quella comunità segua effettivamente tali pratiche di lavoro. Per un osservatore esterno, è molto più difficile rispondere a questa domanda.

- L'ingegneria del software è un atto di equilibrio.

Nella maggior parte dei casi, è illusorio presumere che la raccolta dei requisiti espressi all'inizio del progetto sia l'unico fattore che conta. In effetti, il termine "requisito" è un termine improprio. Suggerisce qualcosa di immutabile, mentre in realtà la maggior parte dei requisiti sono negoziabili. Esistono numerosi vincoli aziendali, tecnici e politici che possono influenzare un progetto di sviluppo software. Ad esempio, si può decidere di utilizzare la tecnologia di database X anziché Y, semplicemente a causa dell'esperienza disponibile con tale tecnologia. In casi estremi, le caratteristiche dei componenti disponibili possono determinare la funzionalità offerta, piuttosto che il contrario.

L'elenco sopra mostra che l'ingegneria del software ha molte sfaccettature. L'ingegneria del software *non* è certamente la stessa cosa della programmazione, sebbene la programmazione sia un ingrediente importante dell'ingegneria del software. Gli aspetti matematici giocano un ruolo poiché ci occupiamo della correttezza del software. Sono necessarie solide pratiche ingegneristiche per produrre prodotti utili. Gli aspetti psicologici e sociologici giocano un ruolo nella comunicazione tra uomo e macchina, organizzazione e macchina e tra umani. Infine, il processo di sviluppo deve essere controllato, che è una questione di gestione.

Il termine "ingegneria del software" allude a possibili somiglianze tra la costruzione di programmi e la costruzione di case o ponti. Questi tipi di somiglianze esistono. In entrambi i casi, lavoriamo da un insieme di funzioni desiderate, utilizzando tecniche scientifiche e ingegneristiche in modo creativo. Le tecniche che sono state applicate con successo nella costruzione di manufatti fisici sono utili anche quando applicate alla costruzione di sistemi software: sviluppo del prodotto in più fasi, pianificazione attenta di queste fasi, controllo continuo dell'intero processo, costruzione da un design chiaro e completo, ecc.

Anche in una disciplina ingegneristica matura, ad esempio nella progettazione di ponti, gli incidenti accadono. I ponti crollano ogni tanto. La maggior parte dei problemi nella progettazione dei ponti si verifica quando i progettisti estrapolano oltre i loro modelli e le loro competenze. Un esempio famoso è il fallimento del Tacoma Narrows Bridge nel 1940. I progettisti di quel ponte hanno estrapolato oltre la loro esperienza per creare travi di irrigidimento più flessibili per un ponte sospeso. Non hanno pensato all'aerodinamica e alla risposta del ponte al vento. Di conseguenza, quel ponte crollò poco dopo essere stato terminato. Questo tipo di estrapolazione sembra essere la regola piuttosto che l'eccezione nello sviluppo del software. Ci impegniamo regolarmente in progetti di sviluppo software che vanno ben oltre la nostra esperienza.

Ci sono anche buone ragioni per considerare la costruzione di software come qualcosa di completamente diverso dalla costruzione di prodotti fisici. Il costo di costruzione del software viene sostenuto durante lo sviluppo e non durante la produzione. La copia del software è quasi gratuita. Il software è di natura logica piuttosto che fisica. I prodotti fisici si consumano nel tempo e quindi devono essere mantenuti. Il software non si consuma. La necessità di mantenere il software è causata da errori rilevati in ritardo o dal cambiamento dei requisiti dell'utente. L'affidabilità del software è determinata dalla manifestazione di errori già presenti, non da fattori fisici come l'usura. Potremmo anche sostenere che il software si esaurisce *perché* viene mantenuto.

Considerare l'ingegneria del software come una branca dell'ingegneria è problematico anche per un altro motivo. La metafora dell'ingegneria allude a un lavoro disciplinato, una pianificazione adeguata, una buona gestione e così via. Suggerisce di affrontare esigenze chiaramente definite, che possono essere soddisfatte se seguiamo tutti i passi giusti. Molti progetti di sviluppo software, tuttavia, implicano la traduzione di alcuni fenomeni del mondo reale in forma digitale. La conoscenza incorporata in questo fenomeno della vita reale è tacita, indefinita, non codificata e potrebbe essersi sviluppata in un lungo periodo di tempo. L'assunto che abbiamo a che fare con un problema ben definito semplicemente non regge. Piuttosto, il processo di progettazione è aperto e la soluzione emerge man mano che procediamo. Questa dicotomia si riflette nel modo in cui la visione dell'ingegneria del software è cambiata nel tempo (Eischen, **2002**). All'inizio, il campo era visto come un mestiere. In una contromossa, è stato coniato il termine "ingegneria del software" e sono stati introdotti molti concetti di fabbrica. Alla fine degli anni '90, il pendolo è tornato indietro e l'aspetto artigianale è stato nuovamente enfatizzato, nel movimento agile (vedi **capitolo 3**). Sia gli aspetti ingegneristici che quelli artigianali hanno il loro posto e daremo un trattamento equilibrato di entrambi.

Due caratteristiche che rendono i progetti di sviluppo software ancora più difficili da gestire sono la visibilità e la continuità. È molto più difficile vedere progressi nella costruzione di software che nella costruzione di un ponte. Si sente spesso la frase che un programma "è quasi finito". Altrettanto spesso si sottovaluta il tempo necessario per finire gli ultimi pezzi.

Questa sindrome "completa al 90%" è molto pervasiva nello sviluppo del software. Non sapendo come misurare il reale progresso, spesso utilizziamo una misura surrogata, il tasso di spesa delle risorse. Ad esempio, un progetto con un budget di 100 giorni persona viene percepito come completo al 50% dopo che sono stati spesi 50 giorni persona. A rigor di termini, confondiamo la velocità con il progresso. A causa della misurazione imprecisa dei progressi e della consueta sottovalutazione dello sforzo totale, i problemi si accumulano con il passare del tempo.

I sistemi fisici sono spesso continui, nel senso che piccoli cambiamenti nelle specifiche portano a piccoli cambiamenti nel prodotto. Questo non è vero con il software. Piccole modifiche alle specifiche del software possono portare a notevoli cambiamenti nel software stesso. Allo stesso modo, piccoli errori nel software possono avere effetti considerevoli. Il razzo spaziale Mariner per Venere, ad esempio, è andato perso a causa di un errore di battitura in un programma FORTRAN. Nel 1998, il Mars Climate Orbiter si è perso perché un team di sviluppo ha utilizzato unità inglesi, come pollici e piedi, mentre un altro team ha utilizzato unità metriche.

Possiamo anche fare un confronto tra ingegneria del software e informatica. L'informatica è emersa come disciplina separata negli anni '60. Si è separato dalla matematica ed è stato fortemente influenzato dalla matematica. Gli argomenti studiati in informatica, come la complessità degli algoritmi, i linguaggi formali e la semantica dei linguaggi di programmazione, hanno un forte sapore matematico. Le tesi di dottorato in informatica contengono invariabilmente teoremi con dimostrazioni di accompagnamento.

Quando il campo dell'ingegneria del software è emerso dall'informatica, ha avuto un'inclinazione simile a concentrarsi su aspetti puliti dello sviluppo del software che possono essere formalizzati, sia nell'insegnamento che nella ricerca. Davamo per scontato che i requisiti potessero essere dichiarati completamente prima dell'inizio del progetto, concentrandoci su sistemi costruiti da zero e ignorando la realtà dello scambio degli aspetti di qualità con il budget disponibile, per non parlare delle trincee della manutenzione del software.

L'ingegneria del software e l'informatica hanno una notevole sovrapposizione. La pratica dell'ingegneria del software, tuttavia, deve anche occuparsi di questioni quali la gestione di grandi progetti di sviluppo, i fattori umani (riguardanti sia il team di sviluppo che i potenziali utenti del sistema) e la stima e il controllo dei costi. Gli ingegneri del software devono *progettare* il software.

L'ingegneria del software ha molte cose in comune con altri campi dell'ingegneria e con l'informatica. Ha anche molte sfaccettature uniche.

1.2 FASI NELLO SVILUPPO DEL SOFTWARE

Quando costruisce una casa, il costruttore non inizia accumulando mattoni. Piuttosto, vengono prima analizzate le esigenze e le possibilità del cliente, tenendo conto di fattori come la struttura familiare, gli hobby, le finanze e così via. L'architetto prende in considerazione questi fattori quando progetta una casa. Solo dopo che il progetto è stato concordato viene avviata la costruzione vera e propria.

È opportuno agire allo stesso modo quando si costruisce un software. Per prima cosa si analizza il problema da risolvere e si descrivono i requisiti in modo molto preciso. Quindi viene realizzato un design basato su questi requisiti. Infine, viene avviato il processo di costruzione, ovvero la programmazione vera e propria della soluzione. Ci sono un numero distinguibile di fasi nello sviluppo del software. Le fasi, discusse in questo libro, sono illustrate nella **Figura 1.2** .

Il **modello di processo** illustrato nella Figura 1.2 è piuttosto semplice. In realtà, le cose di solito saranno più complesse. Ad esempio, la fase di progettazione è spesso suddivisa in una fase di progettazione architettonica globale e una fase di progettazione dettagliata, e spesso si distinguono varie fasi di test. Gli elementi di base, tuttavia, rimangono quelli riportati nella Figura 1.2 . Queste fasi devono essere superate in ogni progetto. A seconda del tipo di progetto e dell'ambiente di lavoro, potrebbe essere necessario uno schema più dettagliato.

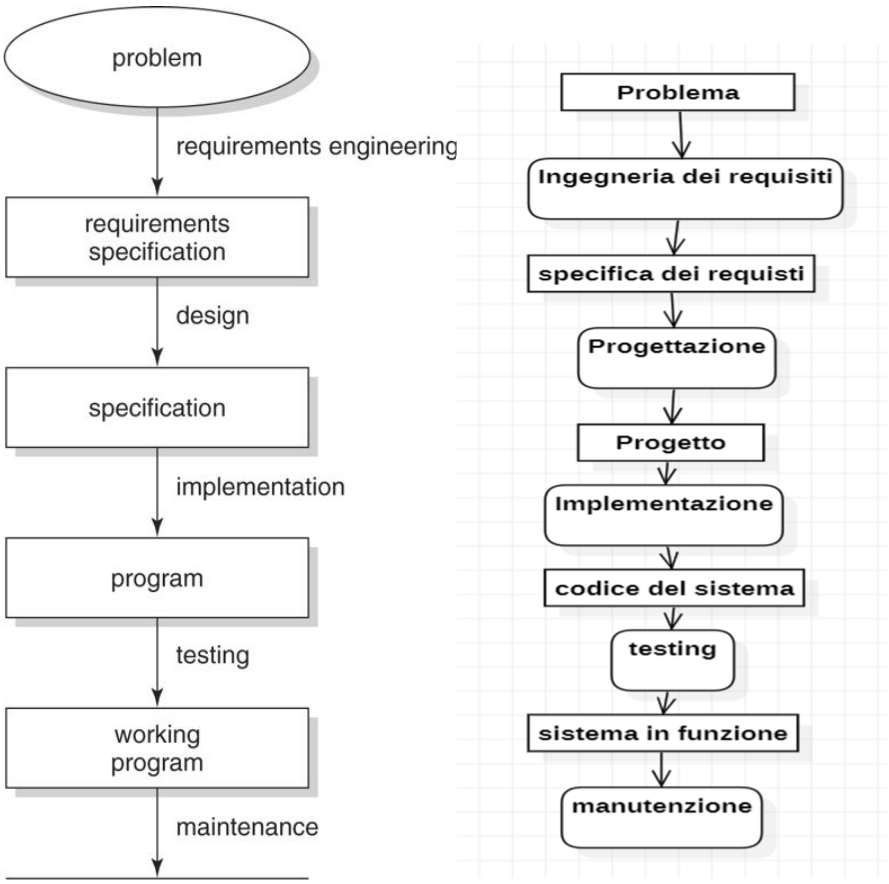


Figura 1.2Una vista semplice dello sviluppo del software – come diagramma UML della attività: i box rettangolari sono gli oggetti, quelli con angoli smussati le attività (nota: vedremo i diagrammi UML meglio più avanti)

Nella **Figura 1.2** , le fasi sono state rappresentate in sequenza. Per un determinato progetto, queste attività non sono necessariamente separate così rigorosamente come indicato qui. Possono, e di solito lo fanno, sovrapporsi. È, ad esempio, del tutto possibile avviare l'implementazione di una parte del sistema mentre alcune delle altre parti non sono state ancora completamente progettate. Come vedremo nella **Sezione 1.3**, non c'è una progressione lineare rigorosa dall'ingegneria dei requisiti alla progettazione, dalla progettazione all'implementazione, ecc. Il ritorno alle fasi precedenti si verifica a causa di errori scoperti o di modifiche dei requisiti. È meglio pensare a queste fasi come a una serie di flussi di lavoro. All'inizio, la maggior parte delle risorse viene spesa per il flusso di lavoro di ingegneria dei requisiti. Successivamente, lo sforzo si sposta sull'implementazione e sui flussi di lavoro di test.

Di seguito, viene fornita una breve descrizione di ciascuno degli elementi di base della **Figura 1.2**. Vari modelli di processo alternativi sono discussi nel **Capitolo 3**. Questi modelli alternativi derivano da una critica giustificata al modello ingenuo illustrato nella **Figura 1.2**. L'unico scopo del nostro modello semplice è quello di fornire un'adeguata strutturazione degli argomenti da affrontare. La fase di manutenzione è ulteriormente discussa nella **Sezione 1.3**. Tutti gli elementi del nostro modello di processo sono trattati in modo molto più elaborato nei capitoli successivi.

Ingegneria dei requisiti: l'obiettivo della fase di ingegneria dei requisiti è ottenere una descrizione completa del problema da risolvere e dei requisiti posti da e sull'ambiente in cui il sistema funzionerà. I requisiti posti dall'ambiente possono includere hardware e software di supporto o il numero di potenziali utenti del sistema da sviluppare. In alternativa, l'analisi dei requisiti può portare a determinati vincoli imposti all'hardware ancora da acquisire o all'organizzazione in cui il sistema deve funzionare. Una descrizione del problema da risolvere include cose come:

- le funzioni del software da sviluppare;
- possibili future estensioni del sistema;
- l'importo e il tipo di documentazione richiesta;
- il tempo di risposta e altri requisiti di prestazione del sistema.

Parte dell'ingegneria dei requisiti è uno studio di **fattibilità.** Lo scopo dello studio di fattibilità è valutare se esiste una soluzione al problema che sia sia economicamente che tecnicamente fattibile.

Più attenti siamo durante la fase di ingegneria dei requisiti, maggiori sono le possibilità che il sistema definitivo soddisfi le aspettative. A tal fine, le varie persone coinvolte (tra gli altri, il

cliente, i potenziali utenti, i progettisti e i programmatori) devono collaborare intensamente. Queste persone hanno spesso background molto diversi, il che non facilita la comunicazione.

Il documento in cui è stabilito il risultato di questa attività è chiamato **specifica dei requisiti.**

Progettazione: Durante la fase di progettazione viene sviluppato un modello dell'intero sistema che, se codificato in qualche linguaggio di programmazione, risolve il problema per l'utente. A tal fine il problema viene scomposto in parti gestibili dette **componenti;** le funzioni di questi componenti e le **interfacce** tra di loro sono specificate in modo molto preciso. La fase di progettazione è cruciale. L'ingegneria dei requisiti e la progettazione sono talvolta visti come una fastidiosa introduzione alla programmazione, che è spesso vista come il vero lavoro. Questo atteggiamento ha un'influenza molto negativa sulla qualità del software risultante.

Le prime decisioni di progettazione hanno un impatto importante sulla qualità del sistema finale. Queste prime decisioni di progettazione possono essere catturate in una descrizione globale del sistema, cioè la sua **architettura.** L'architettura può essere successivamente valutata, fungere da modello per lo sviluppo di una famiglia di sistemi simili o essere utilizzata come scheletro per lo sviluppo di componenti riutilizzabili. In quanto tale, la descrizione dell'architettura di un sistema è un documento importante negli attuali progetti di sviluppo software.

In fase di progettazione si cerca di separare il *cosa* dal *come.* Ci concentriamo sul problema e non dobbiamo lasciarci distrarre da problemi di attuazione.

Il risultato della fase di progettazione, la **specifica (tecnica**), funge da punto di partenza per la fase di implementazione. Se la specificazione è di natura formale, può essere utilizzata anche per derivare prove di correttezza.

Implementazione: Durante la fase di implementazione, ci concentriamo sui singoli componenti. Il nostro punto di partenza è la specifica del componente. Spesso è necessario introdurre un'ulteriore fase di "progettazione", poiché il passaggio dalla specifica del componente al codice eseguibile è spesso troppo ampio. In questi casi, potremmo trarre vantaggio da alcune notazioni di alto livello simili a quelle di un linguaggio di programmazione, come uno **pseudocodice.** (Uno pseudocodice è una specie di linguaggio di programmazione. La sua sintassi e semantica sono generalmente meno rigide, in modo che gli algoritmi possano essere formulati a un livello più alto, più astratto.)

È importante notare che il primo obiettivo di un programmatore dovrebbe essere lo sviluppo di un programma ben documentato, affidabile, di facile lettura, flessibile e corretto. L'obiettivo *non dovrebbe* essere quello di produrre un programma molto efficiente pieno di trucchi. Torniamo alle molteplici dimensioni della qualità del software nel **Capitolo 6**.

Durante la fase di progettazione, viene imposta una struttura globale attraverso l'introduzione dei componenti e delle loro interfacce. Nei linguaggi di programmazione più classici, gran parte di questa struttura tende a perdersi nel passaggio dal design al codice.

Linguaggi di programmazione più recenti offrono possibilità per mantenere questa struttura nel codice finale attraverso il concetto di moduli o classi.

Il risultato della fase di implementazione è un programma eseguibile.

Testing: In realtà, è sbagliato dire che il testing è una fase successiva all'implementazione. Ciò suggerisce che non è necessario preoccuparsi del test fino al termine dell'implementazione. Questo non è vero. È anche giusto dire che questo è uno dei più grandi errori che puoi fare.

Occorre prestare attenzione al testing (in italiano **collaudo**) anche durante la fase di ingegnerizzazione dei requisiti. Durante le fasi successive, il testing viene continuato e affinato. Prima vengono rilevati gli errori, più economico è correggerli.

I test alla fine di ogni fase hanno due obiettivi. Dobbiamo verificare che il passaggio tra le fasi successive sia corretto (questo è noto come **verifica)**. Dobbiamo anche verificare di essere ancora sulla strada giusta per soddisfare i requisiti degli utenti (**convalida)**. Il risultato dell'aggiunta delle attività di verifica e validazione al modello lineare della **Figura 1.2** produce il cosiddetto **modello a cascata** dello sviluppo del software (si veda anche **il Capitolo 3**).

Manutenzione: Dopo la consegna del software, spesso si verificano errori che non sono stati rilevati. Ovviamente, questi errori devono essere riparati. Inoltre, l'uso effettivo del sistema può portare a richieste di modifiche e miglioramenti. Tutti questi tipi di cambiamento sono indicati con il termine piuttosto sfortunato mantenimento. La manutenzione riguarda quindi tutte le attività necessarie per mantenere operativo il sistema dopo che è stato consegnato all'utente.

La gestione dei progetti è un'attività che abbraccia tutte le fasi. Come per altri progetti, i progetti di sviluppo software devono essere gestiti correttamente per garantire che il prodotto venga consegnato in tempo e nel rispetto del budget. Le caratteristiche di visibilità e continuità dello sviluppo del software, nonché il fatto che molti progetti di sviluppo del software vengono intrapresi con un'esperienza pregressa insufficiente, ostacolano seriamente il controllo del progetto. I numerosi esempi di progetti di sviluppo software che non rispettano i tempi previsti forniscono ampie prove del fatto che non abbiamo ancora affrontato in modo soddisfacente questo problema. **I capitoli 2** – **8** trattano gli aspetti principali della gestione dei progetti software, come la pianificazione del progetto, l'organizzazione del team, i problemi di qualità, la stima dei costi e della pianificazione.

Un'attività importante non identificata separatamente è la **documentazione.** Una serie di ingredienti chiave della documentazione di un progetto software sono elaborati nei capitoli che seguono. I componenti chiave della documentazione del sistema includono il piano del progetto, il piano della qualità, la specifica dei requisiti, la descrizione dell'architettura, la documentazione del progetto e il piano di test. Per i progetti più grandi, sarà necessario dedicare un notevole sforzo alla documentazione adeguata del progetto. Lo sforzo di documentazione deve iniziare all'inizio del progetto. In pratica, la documentazione è spesso

vista come un saldo contabile. Poiché molti progetti hanno poco tempo, la documentazione tende ad avere la peggio. I manutentori e gli sviluppatori di software lo sanno e adattano il loro modo di lavorare di conseguenza. Come regola generale, (Lethbridge *et al.*, **2003**) affermano che, più ci si avvicina al codice, più accurata deve essere la documentazione affinché gli ingegneri del software lo utilizzino. Documenti sui requisiti obsoleti e altra documentazione di alto livello possono comunque fornire indizi preziosi. Sono utili per le persone che devono conoscere un nuovo sistema o devono sviluppare casi di test, ad esempio. La documentazione obsoleta di basso livello non ha valore ei programmatori consultano il codice anziché la relativa documentazione. Dal momento che il sistema subirà modifiche dopo la consegna, a causa di errori che non sono stati rilevati o che cambiano i requisiti dell'utente, una documentazione adeguata e aggiornata è di fondamentale importanza durante la manutenzione.

Un elemento particolarmente degno di nota della documentazione è la documentazione dell'utente. Lo sviluppo del software dovrebbe essere orientato alle attività, nel senso che il software da fornire dovrebbe supportare gli utenti nel loro ambiente di attività. Allo stesso modo, la documentazione per l'utente dovrebbe essere orientata alle attività. I manuali utente non dovrebbero solo descrivere le caratteristiche di un sistema, dovrebbero aiutare le persone a fare le cose (Rettig, **1991**). Non possiamo semplicemente fare affidamento sulla struttura dell'interfaccia per organizzare la documentazione dell'utente (proprio come un manuale di riferimento del linguaggio di programmazione non è una fonte appropriata per imparare a programmare).

figura 1.3 fornisce un'indicazione approssimativa dello sforzo relativo solitamente speso per le varie attività fino alla consegna del sistema. Da questi dati emerge un trend molto chiaro, la cosiddetta regola 40–20–40: solo il 20% dello sforzo viene speso per programmare (codificare) effettivamente il sistema, mentre le fasi precedenti (ingegnerizzazione e progettazione dei requisiti) e collaudo ciascuno consuma circa il 40% dello sforzo totale.

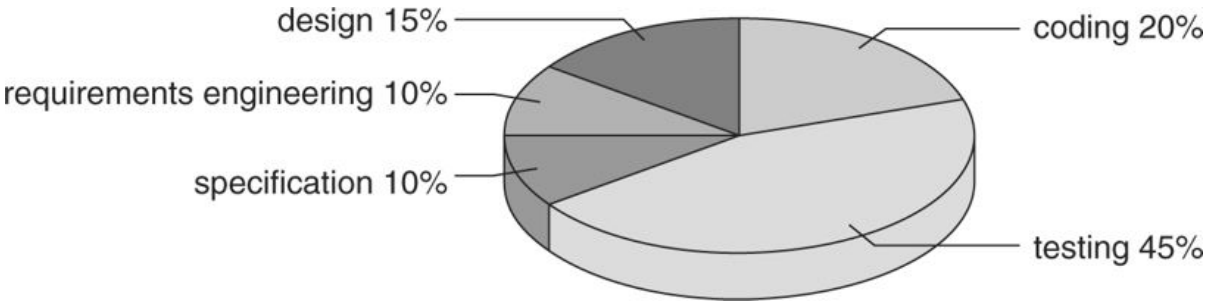


Figura 1.3 Impegno relativo per le varie attività

A seconda delle condizioni al contorno specifiche, delle proprietà del sistema da costruire e così via, si possono trovare variazioni a questa regola. Per i progetti di sviluppo iterativi, ad esempio, la distinzione tra ingegneria dei requisiti, progettazione, implementazione e test

(unità) diventa offuscata. Per la maggior parte dei progetti, tuttavia, questa regola pratica è abbastanza praticabile.

Ciò non implica che la regola 40–20–40 sia quella per cui si deve lottare. Gli errori commessi durante l'ingegneria dei requisiti sono quelli più costosi da riparare (vedere anche **il Capitolo 13**). È molto meglio dedicare più energia alla fase di ingegneria dei requisiti piuttosto che cercare di rimuovere gli errori durante la fase di test che richiede tempo o, peggio ancora, durante la manutenzione. Secondo (Boehm, **1987b**), i progetti di successo seguono uno schema 60–15–25: 60% ingegneria e progettazione dei requisiti, 15% implementazione e 25% test. Il messaggio è chiaro: più a lungo rimandi la codifica, prima hai finito.

Figura 1.3 non mostra l'entità dello sforzo di manutenzione. Quando consideriamo il costo totale di un sistema software nel corso della sua vita, risulta che, in media, la sola manutenzione consuma il 50–75% di questi costi; vedere anche **la Figura 1.1** . Pertanto, la sola manutenzione consuma più delle varie fasi di sviluppo messe insieme.

1.3 MANUTENZIONE O EVOLUZIONE

- *L'unica cosa che manteniamo è la soddisfazione dell'utente*. Lehman (**1980**)

Una volta che il software è stato consegnato, di solito contiene ancora errori che, una volta scoperti, devono essere riparati. Si noti che questo tipo di manutenzione non è causato dall'usura. Piuttosto, riguarda la riparazione di difetti nascosti. Questo tipo di riparazione è paragonabile a quella che si incontra dopo la prima occupazione di una casa di nuova costruzione.

La storia diventa molto diversa se iniziamo a parlare di modifiche o miglioramenti al sistema. La riverniciatura del nostro ufficio o la riparazione di una perdita nel tetto della nostra casa si chiama manutenzione. L'aggiunta di un'ala al nostro ufficio è raramente chiamata manutenzione.

Questo è più di un gioco insignificante con le parole. Durante la vita totale di un sistema software, vengono spesi più soldi per la manutenzione di quel sistema che per lo sviluppo iniziale. Se tutte queste spese riguardassero semplicemente la riparazione di errori commessi durante una delle fasi di sviluppo, la nostra attività andrebbe davvero molto male. Fortunatamente, non è questo il caso.

Distinguiamo quattro tipi di attività di manutenzione:

- **correttiva** – la riparazione degli errori;
- **adattiva:** adattare il software ai cambiamenti nell'ambiente, come il nuovo hardware o la prossima versione di un sistema operativo o di database;
- **perfettiva:** adattare il software a requisiti dell'utente nuovi o modificati, come funzioni extra che devono essere fornite dal sistema, aumentare le prestazioni del sistema o migliorare la sua interfaccia utente;

- **preventiva:** aumentare la futura manutenibilità del sistema, ad esempio aggiornando la documentazione, aggiungendo commenti o migliorando la struttura modulare di un sistema.

Solo la prima categoria può essere giustamente definita manutenzione. Questa categoria, tuttavia, rappresenta solo circa un quarto dello sforzo di manutenzione totale. Approssimativamente un altro quarto dello sforzo di manutenzione riguarda l'adattamento del software ai cambiamenti ambientali, mentre metà del costo di manutenzione viene speso in modifiche per soddisfare le mutevoli esigenze degli utenti, ovvero miglioramenti al sistema (vedere la **Figura 1.4**).

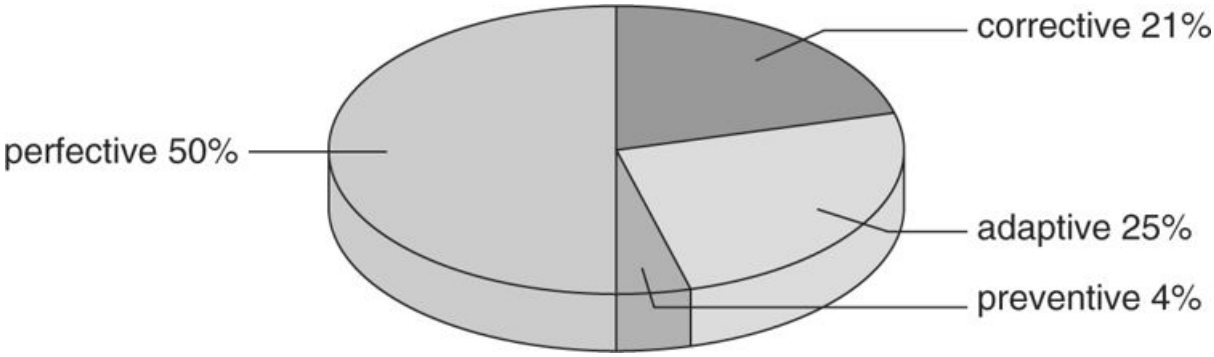


Figura 1.4 Distribuzione delle attività di manutenzione

I cambiamenti sia nell'ambiente del sistema che nei requisiti degli utenti sono inevitabili. Il software modella parte della realtà e la realtà cambia, che ci piaccia o no. Quindi anche il software deve cambiare. Deve *evolversi.* Una grande percentuale di ciò che siamo abituati a chiamare manutenzione è in realtà evoluzione. La manutenzione a causa dei nuovi requisiti dell'utente si verifica sia nei sistemi di alta che di bassa qualità. Un sistema di successo richiede funzionalità nuove e impreviste, a causa del suo utilizzo da parte di molti utenti soddisfatti. Un sistema meno efficace deve essere adattato per soddisfare i suoi clienti.

Il risultato è che il processo di sviluppo del software diventa ciclico, da cui l'espressione ' **ciclo di vita del software** '. Il ritorno alle fasi precedenti, a cui si è accennato in precedenza, non si verifica solo durante la manutenzione. Durante le altre fasi, inoltre, itereremo di volta in volta le fasi precedenti. Durante la progettazione, è possibile che la specifica dei requisiti non sia completa o contenga requisiti contrastanti. Durante i test possono verificarsi errori introdotti nella fase di implementazione o di progettazione. In questi casi e simili è necessaria un'iterazione delle fasi precedenti. Torneremo su questa natura ciclica del processo di sviluppo del software nel **Capitolo 3** , quando discuteremo di modelli alternativi del processo di sviluppo del software.

1.4 DALLA TRINCEA

E tale è la via di ogni superstizione, sia in astrologia, sogni, presagi, giudizi divini o simili; in cui gli uomini, avendo diletto in tali vanità, osservano gli eventi quando sono compiuti, ma quando

falliscono, quantunque ciò avvenga molto più spesso, li trascurano e tralasciano. Ma con molta più sottigliezza questo male si insinua nella filosofia e nelle scienze; in cui la prima conclusione colora e rende conforme a sé tutto ciò che viene dopo, sebbene molto più sano e migliore. Inoltre, indipendentemente da quella gioia e vanità che ho descritto, è l'errore peculiare e perpetuo dell'intelletto umano essere più commosso ed eccitato da affermazioni che da negative; mentre dovrebbe propriamente mantenersi indifferentemente disposto verso l'uno e l'altro allo stesso modo. In effetti, nella definizione di un vero assioma, l'istanza negativa è la più forte delle due. Sir Francis Bacon, Il nuovo Organon, Aforismi XLVI (1611)

*I casi di studio storici contengono una ricchezza di saggezza sulla natura del design e sul metodo ingegneristico. Petroski (**1994**)*

*Gli ingegneri non devono mai presumere che i loro prodotti raggiungeranno la perfezione. Colwell (**2002**)*

Nel suo meraviglioso libro, Henri Petroski ci racconta alcuni dei più grandi successi ingegneristici e, soprattutto, fallimenti di tutti i tempi. Sono apparse anche alcune di queste storie di fallimenti sulla nostra professione. Quattro di loro sono discussi in questa sezione.

Queste storie sono interessanti perché ci insegnano che l'ingegneria del software ha molte sfaccettature. I fallimenti nei progetti di sviluppo software spesso non sono unidimensionali. Non sono causati *solo* da un errore tecnico in qualche routine. Non sono *solo* causati da una cattiva gestione. Non sono *solo* il risultato di problemi di comunicazione umana. Spesso è una combinazione di molti piccoli errori, che si accumulano nel tempo e alla fine si traducono in un grave fallimento. Per parafrasare un famoso detto di Fred Brooks sui progetti che arrivano in ritardo:

- In che modo un progetto finisce davvero nei guai?

Uno scivolone alla volta.

Ciascuna delle storie discusse di seguito mostra un tale effetto cumulativo. I successi nello sviluppo del software non si ottengono se ci limitiamo a impiegare i programmatori più brillanti. Oppure applica la più recente filosofia di design. O avere la più ampia consultazione degli utenti. O anche assumere il miglior manager. Devi fare tutto questo. E anche di più.

1.4.1 Ariane 5, Volo 501

Il volo inaugurale del lanciatore Ariane 5 ha avuto luogo il 4 giugno 1996. Dopo circa 40 secondi, a un'altitudine inferiore a 4 chilometri, il lanciatore si è rotto ed è esploso. Questa perdita di $ 500 milioni è stata in definitiva causata da un overflow nella conversione da un numero a virgola mobile a 64 bit a un intero con segno a 16 bit. Dal punto di vista dell'ingegneria del software, la storia di Ariane 5 è interessante perché il fallimento può essere

attribuito a cause diverse, a diversi livelli di comprensione: test inadeguati, tipo sbagliato di riutilizzo o filosofia di progettazione sbagliata.

L'altitudine del lanciatore e i suoi movimenti nello spazio sono misurati da un sistema di riferimento inerziale (Système de Référence Inertielle, SRI). Ci sono due SRI che operano in parallelo. Il loro hardware e software è identico. La maggior parte dell'hardware e del software per l'SRI è stata conservata dall'Ariane 4. La conversione fatale è avvenuta in un pezzo di software nell'SRI che è significativo solo prima del decollo. Sebbene questa parte del software non serva a nulla dopo il lancio del razzo, continua a funzionare per un numero aggiuntivo di secondi. Questo requisito è stato affermato più di dieci anni prima per una ragione alquanto peculiare. Consente un rapido riavvio del conto alla rovescia, se viene interrotto in prossimità del decollo. Questo requisito non si applica ad Ariane 5, ma il software è rimasto invariato, dopotutto ha funzionato. Poiché l'Ariane 5 è molto più veloce dell'Ariane 4, il razzo raggiunge una velocità orizzontale molto più elevata entro questo breve periodo dopo il decollo, provocando il suddetto overflow. A causa di questo overflow, il primo SRI ha cessato di funzionare. Il secondo SRI è stato quindi attivato, ma poiché l'hardware e il software di entrambi gli SRI sono identici, anche il secondo SRI ha fallito. Di conseguenza, i dati errati sono stati trasmessi dall'SRI al computer di bordo. Sulla base di questi dati errati, sono state comandate le deviazioni complete degli ugelli. Questi hanno causato un carico aerodinamico molto elevato, che ha portato alla separazione dei booster dal razzo principale; questo, a sua volta, ha innescato l'autodistruzione del lanciatore.

Esistono diversi livelli in cui il fallimento di Ariane 5 può essere compreso e spiegato:

- Si è trattato di un errore del software, che avrebbe potuto essere rivelato con test più approfonditi. Questo è vero: il comitato che ha indagato sull'evento è riuscito a smascherare il fallimento utilizzando simulazioni estese.
- L'errore è stato causato dal riutilizzo di un componente difettoso. Anche questo è vero ma, a causa delle caratteristiche fisiche dell'Ariane 4, questo difetto non era mai diventato evidente. C'erano stati molti voli di successo di Ariane 4, utilizzando essenzialmente lo stesso sottosistema SRI. Apparentemente, il riutilizzo non è compositivo: l'uso corretto di un componente in un ambiente non è garanzia per il riutilizzo riuscito di quel componente in un altro ambiente.
- Il fallimento è stato causato da un difetto nel design. Il software Ariane ha seguito una tipica filosofia di progettazione hardware: se un componente si guasta, si presume che la causa sia casuale e viene gestita spegnendo quella parte e invocando un componente di backup. In caso di anomalia del software, che non è casuale, un backup identico è di scarsa utilità. Per la parte software, potrebbe essere stata seguita una linea diversa. Ad esempio, si sarebbe potuto chiedere al componente di fornire la migliore stima delle informazioni richieste.

1.4.2 Therac-25

Il Therac-25 è una macchina per radiazioni controllata da computer. Ha tre modalità:

- Modalità luce di campo. Questa posizione facilita semplicemente il corretto posizionamento del paziente.
- Modalità elettronica. Nella terapia elettronica, il computer controlla l'energia e la corrente (variabile) del raggio e i magneti diffondono il raggio a una concentrazione sicura.
- Modalità fotonica (raggi X). In modalità fotone, l'energia del raggio è fissa. Tra l'acceleratore e il paziente viene interposto un 'appiattimento del raggio' per produrre un campo di trattamento uniforme. Una corrente molto elevata (circa 100 volte superiore a quella della modalità elettronica) è necessaria su un lato dello spianatore del fascio per produrre una dose di trattamento ragionevole sull'altro lato.

La macchina è dotata di una piattaforma girevole che ruota l'attrezzatura necessaria in posizione. La situazione di pericolo di base è ovvia da quanto sopra: un raggio di fotoni viene emesso dall'acceleratore mentre lo spianatore del raggio non è in posizione. Il paziente viene quindi trattato con una dose troppo elevata. Questo è successo più volte. Di conseguenza, diversi pazienti sono morti e altri sono rimasti gravemente feriti.

Uno dei malfunzionamenti del Therac-25 è diventato noto come "Malfunction 54". Un paziente è stato preparato per il trattamento. L'operatore ha digitato i dati necessari sulla console in una stanza adiacente. Mentre lo faceva, ha commesso un errore: ha digitato "x" (per la modalità a raggi X) invece di "e" (per la modalità elettronica). Ha corretto il suo errore spostando il cursore sul campo appropriato, digitando il codice corretto e premendo più volte il tasto Invio fino a quando il cursore era di nuovo sulla riga di comando. Quindi ha premuto 'B' (raggio acceso). La macchina si è fermata ed ha emesso il messaggio 'Malfunzionamento 54'. Questo particolare messaggio di errore indica una dose errata, troppo alta o troppo bassa. La console ha indicato un sostanziale sottodosaggio. L'operatore sapeva che spesso la macchina presentava delle stranezze e che di solito potevano essere risolte semplicemente premendo "P" (procedi). Così ha fatto. Lo stesso messaggio di errore è apparso di nuovo. Normalmente, l'operatore avrebbe un contatto audio e video con il paziente nella sala di trattamento. Non questa volta, però: l'audio era rotto e il video era stato spento. Successivamente è stato stimato che il paziente avesse ricevuto 16.000-25.000 rad su una superficie molto piccola, invece della dose prevista di 180 rad. Il paziente si ammalò gravemente e morì cinque mesi dopo.

La causa di questo evento pericoloso è stata fatta risalire al software che gestisce la macchina per radiazioni. Dopo che l'operatore ha terminato l'immissione dei dati, potrebbe iniziare l'impostazione fisica della macchina. La flessione dei magneti dura circa otto secondi. Dopo che i magneti sono stati posizionati, controlla di nuovo se qualcosa è cambiato. Se l'operatore riesce ad apportare modifiche **e** riportare il cursore nella posizione della riga di comando

entro gli otto secondi necessari per impostare i magneti, ci saranno modifiche nei parametri interni del sistema ma il sistema comunque 'penserà' che non è successo nulla e semplicemente continuare, con le conseguenze sopra descritte.

Incidenti come questo vengono segnalati alla Federal Drug Administration (FDA). La FDA ha chiesto al produttore di adottare le misure appropriate. La "correzione" suggerita era la seguente:

- Con effetto immediato, e fino a nuovo avviso, il tasto utilizzato per riportare indietro il cursore nella sequenza di prescrizione (es. cursore 'UP' inscritto con una freccia rivolta verso l'alto) non deve essere utilizzato per la modifica o per qualsiasi altro scopo.
- Per evitare l'uso accidentale di questa chiave, è necessario rimuovere il cappuccio della chiave e fissare i contatti dell'interruttore in posizione aperta con nastro isolante o altro materiale isolante....
- La disabilitazione di questo tasto significa che se i dati della prescrizione inseriti non sono corretti, è necessario utilizzare un comando di ripristino 'R' e reinserire l'intera prescrizione.

La FDA non ha accettato questo rimedio. In particolare, hanno giudicato il tono della notifica non commisurato all'urgenza di farlo. La discussione tra la FDA e il produttore è continuata per un po' di tempo prima che fosse data una risposta adeguata a questo e ad altri guasti del Therac-25.

La macchina Therac-25 e il suo software si sono evoluti da modelli precedenti meno sofisticati. Nelle versioni precedenti del software, ad esempio, non era possibile spostarsi su e giù per lo schermo per modificare i singoli campi. Gli operatori hanno notato che trattamenti diversi spesso richiedevano quasi gli stessi dati, che dovevano essere inseriti nuovamente. Per migliorare l'usabilità, è stata aggiunta la funzione per spostare il cursore e modificare i singoli campi. Pertanto, è evidente che la facilità d'uso può entrare in conflitto con la sicurezza.

Anche nei modelli precedenti, la corretta posizione del giradischi e delle altre apparecchiature era assicurata da semplici interblocchi elettromeccanici. Questi interblocchi sono un meccanismo comune per garantire la sicurezza. Ad esempio, vengono utilizzati negli ascensori per assicurarsi che le porte non possano essere aperte se l'ascensore è tra i piani. Nel Therac-25, questi dispositivi di sicurezza meccanici sono stati sostituiti da software. Il software è diventato così un unico punto di errore. Questa eccessiva fiducia nel software ha contribuito agli incidenti del Therac-25, insieme a pratiche di ingegneria del software inadeguate e una reazione inadeguata della gestione agli incidenti.

1.4.3 Il servizio di ambulanza londinese

Il London Ambulance Service (LAS) gestisce il traffico delle ambulanze nella Greater London. Copre un'area di oltre 600 miglia quadrate e trasporta oltre 5 000 pazienti al giorno in 750

veicoli. Il LAS riceve oltre 2 000 telefonate al giorno, comprese più di 1 300 chiamate di emergenza. Il sistema di cui discutiamo qui è un sistema CAD (Computer Aided Dispatch). Tale sistema CAD ha le seguenti funzionalità:

- Risponde alle chiamate, accetta e verifica i dettagli dell'incidente, inclusa la posizione dell'incidente.
- Determina quale ambulanza inviare.
- Mobilita l'ambulanza e comunica i dettagli dell'incidente all'ambulanza.
- Si occupa della gestione delle risorse delle ambulanze, in particolare del posizionamento dei veicoli per ridurre al minimo i tempi di risposta.

Un vero e proprio sistema CAD è piuttosto complesso. In preda al panico, qualcuno potrebbe chiamare e dire che è successo un incidente davanti a Foyle, supponendo che tutti sappiano dove si trova questa libreria. Un'ampia componente di dizionario geografico, inclusa l'identificazione del telefono pubblico, aiuta a risolvere questo tipo di problema. Il sistema CAD contiene anche un sistema radio, terminali mobili nelle ambulanze e un sistema automatico di localizzazione del veicolo.

Il progetto CAD del London Ambulance Service è stato avviato nell'autunno del 1990. La consegna era prevista per gennaio 1992. A quel tempo, tuttavia, il software era ancora lontano dall'essere completo. Nei primi nove mesi del 1992, il sistema è stato installato in modo frammentario in diverse divisioni LAS, ma non è mai stato stabile. Il 26 e 27 ottobre 1992 si sono verificati seri problemi con il sistema e si è deciso di tornare a una modalità di funzionamento parzialmente manuale. Il 4 novembre 1992 il sistema si è bloccato. L'Autorità sanitaria regionale ha istituito una squadra d'inchiesta per indagare sui fallimenti e sulla storia che li ha portati. Hanno tirato fuori un rapporto di 80 pagine, che sembra un romanzo di suspense. Di seguito, si evidenziano alcune delle questioni sollevate in questo rapporto.

Il sistema CAD previsto sarebbe un'impresa importante. Nessun altro servizio di emergenza aveva tentato di arrivare così lontano. Il piano era di passare da un processo completamente manuale, in cui i moduli venivano compilati e trasportati da un dipendente all'altro tramite un nastro trasportatore, all'automazione completa, in un colpo solo. Lo schema era molto ambizioso. I partecipanti sembrano non essersi resi pienamente conto dei rischi che stavano correndo.

Molto prima che il progetto iniziasse effettivamente, è stato chiesto consiglio a una società di consulenza gestionale. Hanno suggerito che una soluzione confezionata costerebbe 1,5 *milioni* di dollari e richiederebbe 19 mesi. Il loro rapporto affermava anche che se non fosse stato possibile trovare una soluzione pacchetto, le stime dovrebbero essere notevolmente aumentate. Alla fine è stata scelta una soluzione non confezionata, ma sono stati ricordati solo i numeri di questo rapporto, o almeno così sembra.

L'annuncio ha prodotto risposte da 35 aziende. Le specifiche e il calendario sono stati discussi con queste società. Il calendario proposto era di 11 mesi (questo non è un errore di battitura).

Sebbene molti fornitori abbiano espresso dubbi sulla tempistica, è stato detto loro che non era negoziabile. Alla fine, 17 fornitori hanno fornito proposte complete. È stata selezionata l'offerta più bassa, a circa $ 1 milione. Questa gara è stata di circa $ 700 000 in meno rispetto all'offerta più bassa successiva. Nessuno sembra aver messo in dubbio questa enorme differenza. La proposta selezionata in modo superficiale suggeriva che l'azienda avesse esperienza nella progettazione di sistemi per i servizi di emergenza. Questa non era una bugia: avevano sviluppato sistemi amministrativi per tali servizi. Il sistema LAS era anche molto più grande di qualsiasi cosa avessero gestito in precedenza.

Il sistema proposto avrebbe un impatto abbastanza significativo sul modo in cui gli equipaggi delle ambulanze svolgevano il loro lavoro. Sarebbe quindi fondamentale avere la loro piena collaborazione. Se gli equipaggi non hanno premuto i pulsanti giusti al momento giusto e nell'ordine giusto, potrebbe nascere il caos. Tuttavia, il coinvolgimento dell'utente durante il processo di ingegneria dei requisiti è stato minimo.

Il sistema CAD previsto funzionerebbe in modo assolutamente obiettivo e imparziale e mobiliterebbe sempre la risorsa ottimale per qualsiasi incidente. Ciò supererebbe molte delle pratiche di lavoro allora attuali che la direzione considerava antiquate e non nell'interesse di LAS. Ad esempio, il nuovo sistema assegnerebbe la risorsa disponibile più vicina indipendentemente dalla stazione di origine. Potrebbe verificarsi il seguente scenario:

- L'equipaggio di John deve subire un incidente a poche miglia a est della loro base.
- Una volta lì, vengono indirizzati in un ospedale a poche miglia più a est per consegnare il paziente.
- Arriva un'altra chiamata e John sembra essere il più vicino. Gli viene ordinato di viaggiare ancora alcune miglia più a est.

In questo modo, gli equipaggi potrebbero dover operare sempre più lontano dalla loro base e in un territorio sconosciuto. Perdono tempo, perché sbagliano strada, o addirittura devono fermarsi per chiedere indicazioni. Hanno anche più da viaggiare per raggiungere la loro stazione di casa alla fine di un turno. Agli equipaggi non è piaciuto questo aspetto del nuovo sistema.

Il nuovo sistema ha anche eliminato la flessibilità delle stazioni di emergenza locali nel decidere quale risorsa allocare. Nel nuovo schema, la gestione delle risorse era completamente centralizzata e gestita dal sistema. Quindi, supponiamo che John corra giù dove sono parcheggiate le ambulanze e il computer gli abbia ordinato di prendere l'auto numero 5. John ha fretta e forse non riesce a individuare rapidamente l'auto numero 5, o forse è parcheggiata dietro alcune altre auto. John sta pensando al paziente che lo aspetta e decide di prendere invece l'auto numero 4. Questo significa guai.

Le persone responsabili dei requisiti sono state fuorviate, o ingenue, nel credere che i sistemi informatici di per sé possano determinare cambiamenti nelle pratiche umane. I computer

sono lì per aiutare le persone a svolgere il proprio lavoro, non viceversa. Le camicie di forza operative sono destinate a fallire.

L'eventuale arresto anomalo del 4 novembre 1992 è stato causato da un piccolo errore di programmazione. Circa tre settimane prima, un programmatore stava lavorando su una parte del sistema e si era dimenticato di rimuovere un piccolo pezzo di testo del programma. Il codice di per sé non ha causato danni. Tuttavia, allocava una piccola quantità di memoria ogni volta che il sistema generava la mobilitazione di un veicolo. Questa memoria non è stata deallocata. Dopo tre settimane, tutta la memoria è stata esaurita e il sistema è andato in crash.

Il progetto LAS nel suo insieme non ha fallito a causa di questo errore del programmatore. Quella era solo l'ultima goccia. Il programma del progetto era troppo stretto. Sia la direzione del London Ambulance Service che l'appaltatore avevano poca o nessuna esperienza con progetti di sviluppo software di queste dimensioni e complessità. Erano troppo ottimisti nella valutazione dei rischi. Presumevano che tutte le persone che avrebbero interagito con il sistema lo avrebbero fatto esattamente nel modo giusto, sempre. Presumevano che le parti hardware del sistema avrebbero funzionato esattamente come specificato. La direzione ha deciso la funzionalità del sistema, senza quasi nessuna consultazione con le persone che sarebbero state i suoi utenti primari. Qualsiasi progetto con tali caratteristiche è destinato a fallire. Fin dal primo giorno.

1.4.4 Chi conta i voti?

- *Non è chi vota che conta, è chi conta i voti*.
  Stalin

I sistemi elettorali tradizionali e non automatizzati lasciano una traccia cartacea che può essere utilizzata a fini di controllo: sono stati contati tutti i voti? sono stati contati correttamente? Tale audit è svolto da una parte indipendente. Queste salvaguardie servono a creare fiducia nel risultato.

Ma cosa accadrebbe se le elezioni fossero supportate dai computer? Come elettore, premi semplicemente un pulsante. Ma cosa succede dopo? La registrazione e il conteggio sono nascosti. Come fai a sapere che il tuo voto non è alterato? Come si possono evitare le frodi? Per l'individuo è necessaria una scheda elettorale, un pezzo di carta simile allo scontrino di un bancomat, che serva a verificare la scelta dell'elettore. Le schede di tutti gli elettori possono essere utilizzate in una verifica indipendente dell'esito delle elezioni. La maggior parte dei sistemi elettorali automatizzati oggi non fornisce queste salvaguardie.

E se facciamo un ulteriore passo avanti e forniamo ai nostri elettori un'applicazione Web per inserire i loro voti? Di seguito la storia di un vero e proprio sistema di questo tipo. L'applicazione è stata sviluppata in Java. A causa delle normative governative, il modello di voto implementato imitava quello tradizionale. L'applicazione mantiene un registro delle

votazioni, contenente l'identificazione di tutti gli elettori, e un'urna in cui sono archiviati i voti. Una delle regole a cui il sistema doveva attenersi è l'anonimato: un voto nelle urne non deve essere riconducibile a un nome nell'elenco delle votazioni. Un'altra normativa riguarda la sicurezza: entrambi i registri devono essere conservati separatamente.

Il progetto tecnico prevedeva due banche dati separate, una per gli elettori e una per le schede elettorali. Mettere un voto e contrassegnare un elettore come "ha votato" dovrebbe essere eseguito in un'unica transazione: entrambe le azioni vengono eseguite o nessuna delle due. Questo design soddisferebbe il requisito di correttezza: il numero di voti nell'urna è uguale al numero di elettori contrassegnati come "ha votato".

Almeno, questo è ciò che speravamo. I test del sistema, tuttavia, hanno mostrato che, apparentemente in momenti casuali, c'erano più voti nelle urne di quanti fossero gli elettori contrassegnati come "ha votato". Quindi il sistema consentiva agli elettori più di un voto.

Dando un'occhiata sotto il cofano, è stato rivelato un errore di codifica nel processo di votazione. Parte dell'algoritmo funzionava come segue:

- 1. Identificare l'elettore.
- 2. Abbina l'elettore a una voce nel registro.
- 3. Se viene trovata una corrispondenza, verificare che l'elettore non abbia ancora votato.

Il test nell'ultimo passaggio aveva la forma

```
voter.getIdentification() == identification()
```

invece di

```
equals(voter.getIdentification(),identification())
```

In altre parole, sono stati confrontati i riferimenti, piuttosto che i valori effettivi. Questo è un modo per vincere le elezioni.

1.5 ETICA DELL'INGEGNERIA DEL SOFTWARE

Supponiamo di testare parte di un grande sistema software. Trovi un bel po' di errori e di certo non sei pronto per la consegna. Tuttavia, il tuo manager ti sta pressando. Il programma è già slittato di alcune settimane. Il tuo manager a sua volta è pressato dal suo capo. Il cliente attende con impazienza la consegna del sistema. Il tuo manager suggerisce di consegnare il sistema così com'è, continuare a testare e sostituire il sistema con una versione migliore entro il mese successivo. Come reagiresti a questo schema? Ti arrenderesti semplicemente? Discuti con il tuo manager? Vai dal suo capo? Vai dal cliente?

Lo sviluppo di sistemi software complessi coinvolge molte persone: sviluppatori software, tester, direttori tecnici, direttori generali, clienti, ecc. All'interno di questa organizzazione temporanea, il rapporto tra gli individui è spesso asimmetrico: una persona che partecipa alla

relazione ha più conoscenza di qualcosa che l'altro. Ad esempio, uno sviluppatore di software ha più conoscenze sul sistema in costruzione rispetto al suo manager. Una relazione così asimmetrica richiede fiducia: se lo sviluppatore dice che lo sviluppo di qualche componente è nei tempi previsti, il suo manager non può non credere a questo messaggio. Almeno per un po. Tale dipendenza offre opportunità per comportamenti non etici, come l'appropriazione indebita, a maggior ragione se esiste anche una relazione di potere tra gli individui.

Non sorprende quindi che le persone all'interno della comunità dell'ingegneria del software abbiano discusso di un **codice etico dell'ingegneria del software.** Due grandi organizzazioni di professionisti del settore, IEEE Computer Society e ACM, hanno sviluppato congiuntamente un codice di questo tipo. La versione breve di questo codice è riportata nella Figura 1.5.

Nella versione lunga del codice, ciascuno dei principi è ulteriormente perfezionato in un insieme di clausole. Alcune di queste clausole sono dichiarazioni di aspirazione: ad esempio, un software engineer dovrebbe sforzarsi di comprendere appieno le specifiche del software su cui lavora. Le aspirazioni dirigono il comportamento professionale. Richiedono un giudizio etico significativo. Altre clausole esprimono obblighi dei professionisti in genere: ad esempio, un ingegnere informatico dovrebbe, come qualsiasi altro professionista, prestare servizio solo nelle aree di sua competenza. Un terzo tipo di clausola è rivolto a comportamenti professionali specifici all'interno dell'ingegneria del software: ad esempio, un ingegnere del software dovrebbe garantire stime realistiche del costo e della pianificazione di qualsiasi progetto su cui lavora.

Ci sono una serie di clausole in questo codice che riguardano la situazione del tester sopra menzionato:

- Approvare il software solo se si ha la convinzione fondata che sia sicuro, soddisfi le specifiche, superi i test appropriati e non riduca la qualità della vita o della privacy o danneggi l'ambiente (clausola 1.03 [1]).
- Garantire test, debug e revisione adeguati del software e dei relativi documenti su cui si lavora (clausola 3.10).
- In qualità di manager, non chiedere a un ingegnere del software di fare nulla di incoerente con questo codice etico (clausola 5.11).
- Sii preciso nell'indicare le caratteristiche del software su cui lavori, evitando non solo affermazioni false, ma anche affermazioni che potrebbero essere ritenute speculative, vacue, ingannevoli, fuorvianti o dubbie (clausola 6.07).

Preamble

The short version of the code summarizes aspirations at a high level of abstraction. The clauses that are included in the full version give exan1ples and details of how' these aspirations change the way we act as software engineering professionals.

Without the aspirations, the details can become legalistic and tedious; Without the details, the aspirations can become high-sounding but empty; together, the aspirations and the details form a cohesive code.

Software engineers shall commit thcn1sclvcs to making the analysis, specification, design, development, testing, and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following eight Principles:

I. **Public**. Software engineers shall act consistently with the public interest.

2. **Client and employer**. Software engineers shall act in a manner that is in the best interests of their client and employer, consistent with the public interest.

3. **Product**. Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.

4. **Judgment**. Software engineers shall maintain integrity and independence in their professional judgment.

5. **Management**. Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.

6. **Profession**. Software engineers sl1all advance the integrity and reputation of the profession consistent with the public interest.

7. **Colleagues**. Software engineers shall be fair to and supportive of their colleagues.

8. **Self**. Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

Figura 1.5 Codice etico di ingegneria del software (ACM/IEEE)

Il codice non è un semplice algoritmo per discriminare tra comportamenti accettabili e inaccettabili. Piuttosto, i principi enunciati dovrebbero influenzare te, come ingegnere del software, a considerare chi è influenzato dal tuo lavoro. Il software che sviluppi colpisce il pubblico. La salute, la sicurezza e il benessere del pubblico è la preoccupazione principale di

questo codice etico. Aderire a questo, o a un codice etico simile, non è solo qualcosa da considerare un venerdì pomeriggio. Dovrebbe diventare uno stile di vita.

Il codice non si rivolge solo agli ingegneri del software. Si rivolge anche ai manager, in quanto il codice indica cosa ci si potrebbe ragionevolmente aspettare da ingegneri del software professionisti.

1.6 QUO VADIS?

Negli ultimi 30 anni sono stati fatti molti progressi. Per ciascuna delle fasi principali sono state sviluppate numerose tecniche e strumenti. Alcuni di questi hanno trovato un uso diffuso. Nella loro valutazione delle pratiche di progettazione e codifica, ad esempio, DeMarco e Lister hanno scoperto che una serie di tecniche ampiamente acclamate (come l'uso di piccole unità, l'associazione di componenti forti e la programmazione strutturata) vengono applicate nella pratica e danno anche i loro frutti (DeMarco e Lister, **1989**). Tuttavia, i brevi schizzi nella **Sezione 1.4** (e la discussione più elaborata nei capitoli seguenti) mostrano che sono ancora necessarie molte ricerche per trasformare l'ingegneria del software in una disciplina ingegneristica veramente matura.

Ci vuole del tempo prima che la tecnologia sviluppata nei laboratori di ricerca venga applicata in modo sistematico. Questo vale per i prodotti fisici, come il transistor, ma anche per i metodi, le tecniche e gli strumenti nell'area della tecnologia software. La prima versione del sistema operativo UNIX risale al 1971; solo dalla fine degli anni '80, l'interesse per UNIX si è ampiamente diffuso. Gli studi sul costo del software furono condotti per la prima volta all'inizio degli anni '60; negli anni '80 c'è stato un crescente interesse per i modelli quantitativi per la stima dei costi del software (si veda anche **il Capitolo 7**). L'articolo di Dijkstra sulla programmazione come attività umana è apparso nel 1965. Alla fine degli anni '70 furono pubblicati i primi libri di testo introduttivi sulla programmazione strutturata. Il termine "ingegneria del software" è stato introdotto nel 1968. Negli anni '80 sono stati avviati ampi programmi nazionali e internazionali per favorire la transizione di questa nuova tecnologia. L'elenco di cui sopra può essere esteso con molti altri esempi (Redwine and Riddle, **1985**). Questo processo di maturazione richiede generalmente almeno 10-15 anni.

In un articolo fondamentale, Brooks (**1987**) discute una serie di approcci potenzialmente fruttuosi per aumentare notevolmente la produttività del software. La sua conclusione principale è che non esiste una pallottola d'argento. Ma non dobbiamo nemmeno aver paura del lupo mannaro. Attraverso un attento studio delle numerose innovazioni e un'indagine sui loro veri meriti, è possibile ottenere molti miglioramenti sia in termini di qualità che di produttività. Il resto di questo testo è dedicato a una valutazione critica di questi sviluppi tecnologici e non.

Diversi sviluppi relativamente recenti hanno avuto un impatto drammatico sul campo:

- L'ascesa dei **metodi agili.** Come notato in precedenza in questo capitolo, il termine "ingegneria del software" implica un approccio ordinato, simile a quello di una fabbrica, allo sviluppo del software. Questo ignora il fatto che, per molti progetti, è impossibile dichiarare i requisiti in anticipo. Emergono mentre andiamo avanti. Armor (**2001**) confronta lo sviluppo software tradizionale con l'abbattimento di uno Zeppelin e gli approcci agili con l'abbattimento di un aereo supersonico. Per abbattere uno Zeppelin, raccogliamo informazioni su altitudine, distanza, velocità e così via, trasmettiamo queste informazioni alla pistola, miriamo e spariamo. Questo approccio non funziona per gli aerei supersonici. Non sappiamo dove sarà l'intercettazione e il missile dovrà cambiare direzione mentre è in aria. È una sfida cercare di combinare con successo approcci di tipo ingegneristico e artigianale allo sviluppo del software.

- Il passaggio dalla *produzione* di software *all'utilizzo del* software. Il time-to-market, i costi e l'assoluta complessità incoraggiano le organizzazioni ad assemblare i sistemi da componenti esistenti, piuttosto che sviluppare quei componenti da zero. I costruttori creano (pezzi di) software e gli integratori integrano quei pezzi nelle applicazioni degli utenti finali. Di conseguenza, i consumatori di software spesso non parlano più con gli sviluppatori. I requisiti provengono da una varietà di altre fonti, come l'analisi del registro delle chiamate dell'helpdesk o ricerche di mercato (Sawyer, **2001**). Per il consumatore, il processo di sviluppo del software non è più interessante, conta solo il prodotto risultante. Questo cambiamento ha dato origine a nuovi argomenti all'interno dell'ingegneria del software, come lo sviluppo di software basato su componenti (CBSD), i componenti commerciali pronti all'uso (COTS), le linee di prodotti software e i servizi.

- Il successo del software open source. Il software open source è un software il cui codice sorgente è disponibile gratuitamente. La maggior parte del software sviluppato è un software closed source, sviluppato allo scopo di venderlo a un cliente. Non è nell'interesse dell'organizzazione in via di sviluppo o del cliente rendere il software disponibile ad altri per la lettura e la modifica. Con il software open source, il codice sorgente *è* disponibile.
  I singoli sviluppatori di software di solito vengono coinvolti nello sviluppo dell'open source per motivi personali: hobby, maggiore stima da parte dei colleghi, opportunità di lavorare su compiti impegnativi, coinvolgimento in progetti ad alta visibilità o migliori opportunità di carriera. Ma questo non basta a spiegare il successo dei progetti open source.

Il successo di prodotti open source come Linux, Apache e Mozilla è in parte causato dagli effetti di rete. Questi prodotti coinvolgono un numero enorme di sviluppatori che si scambiano idee e propongono nuove funzionalità. Le proposte di successo filtrano attraverso i livelli dell'organizzazione di sviluppo open source (vedi **Sezione 5.2.6**) e alla fine vengono accettate dal team di sviluppo principale. Quindi, anziché avere un costoso processo di

ingegneria dei requisiti con un esito rischioso, i requisiti *emergono* come sottoprodotto del processo di sviluppo. Questo è anche noto come *crowdsourcing:* la "folla" è la fonte dei requisiti. La folla di persone coinvolte filtra i requisiti e solo quelli ampiamente accettati riescono a farcela. Il risultato finale è un prodotto le cui caratteristiche sono apprezzate da una vasta comunità di utenti. Molte organizzazioni, ad esempio quelle del mondo delle telecomunicazioni, hanno riconosciuto le opportunità di questo modello di sviluppo e osservano attentamente gli sviluppi nelle comunità open source a cui sono interessate. Fanno accordi strategici con le comunità che ritengono interessanti. A lungo termine, è probabile che questo aspetto di creazione di valore dello sviluppo di software open source sia più importante per il suo successo rispetto all'aspetto del volontariato.

- La natura più eterogenea dello sviluppo del software. In passato, un'organizzazione di sviluppo software aveva tutto sotto controllo. O almeno così pensava. Al giorno d'oggi, il software è sviluppato da team sparsi in tutto il mondo. Parte dello sviluppo può essere esternalizzata a un'organizzazione diversa. Il software incorpora componenti acquistati da altri fornitori, prodotti open source o servizi trovati sul Web. Di conseguenza, un'organizzazione non ha più il controllo.

Per chiudere questo capitolo c'è un elenco di importanti periodici che contengono materiale rilevante per il campo dell'ingegneria del software:

- *Transactions on Software Engineering* (IEEE) è un periodico mensile in cui vengono riportati i risultati della ricerca.
- *Software* (IEEE) è una rivista bimestrale di portata un po' più generale.
- *Software Engineering Notes* (ACM) è una newsletter bimestrale dell'ACM Special Interest Group on Software Engineering.
- *Transactions on Software Engineering and Methodology* (ACM) è una rivista trimestrale che riporta i risultati della ricerca.
- *Il Journal of Systems and Software* (Elsevier) è una rivista mensile che copre sia documenti di ricerca che resoconti di esperienze pratiche.
- *Atti dell'International Conference on Software Engineering* (ACM/IEEE) riporta gli atti della più importante conferenza internazionale del settore, organizzata ogni anno.
- Gli atti della Conferenza internazionale sulla manutenzione del software (IEEE) riportano gli atti di una conferenza annuale.
- *Software Maintenance and Evolution: Research and Practice* (Wiley) è una rivista bimestrale dedicata ad argomenti relativi alla manutenzione e all'evoluzione del software.

1.7 COME CAMBIERA' L'INGEGNERIA DEL SOFTWARE CON L'INTELLIGENZA ARTIFICIALE GENERATIVA?

L'ingegneria del software non scomparirà: cambierà il suo centro di gravità. Scrivere codice diventerà più veloce e meno distintivo; definirne correttamente lo scopo, verificarlo e integrarlo in sistemi reali diventerà più importante.

Area	**Evoluzione probabile**
Scrittura del codice	L'AI genererà funzioni, test, documentazione e refactoring da descrizioni in linguaggio naturale.
Ruolo dello sviluppatore	Da "autore di ogni riga" a progettista, supervisore, revisore e orchestratore di agenti AI.
Requisiti	Sarà essenziale saper formulare specifiche precise, casi limite e criteri di accettazione.
Testing e sicurezza	Aumenteranno test automatici, analisi statica, simulazioni e verifiche formali: il codice generato va comunque controllato.
Architettura	Diventeranno centrali confini tra servizi, dati, autorizzazioni, osservabilità, costi e resilienza.
Produttività	I team piccoli potranno realizzare prodotti più complessi, ma aumenterà anche il rischio di produrre rapidamente software inutile o fragile.
Formazione	Meno memorizzazione sintattica; più comprensione di algoritmi, sistemi distribuiti, sicurezza, dati e dominio applicativo.

Le competenze più preziose saranno:

- ragionamento sui requisiti e sul dominio;
- progettazione di architetture sostenibili;
- capacità di leggere e criticare codice;
- debugging e analisi dei problemi;
- sicurezza, privacy e gestione dei dati;
- comunicazione con utenti, clienti e team;
- valutazione dei compromessi tra qualità, velocità e costo.

Il principale rischio è la “competenza apparente”: un sistema può compilare e sembrare corretto, ma contenere vulnerabilità, errori concettuali o dipendenze ingestibili. Per questo l’AI aumenterà il valore delle pratiche ingegneristiche rigorose, non le renderà superflue.

In sintesi: l’AI generativa automatizzerà molta produzione di codice, ma renderà ancora più importante decidere che cosa costruire, perché costruirlo e come dimostrare che funziona davvero.

1.8 RIEPILOGO

L'ingegneria del software si occupa dei problemi che hanno a che fare con la costruzione di programmi di *grandi dimensioni.* Quando si sviluppano tali programmi, viene seguito un approccio graduale. In primo luogo, il problema viene analizzato, quindi il sistema viene progettato, implementato e testato. Questa pratica ha molto in comune con l'ingegneria dei prodotti fisici; da qui il termine "ingegneria del software". L'ingegneria del software, tuttavia, differisce anche dall'ingegneria dei prodotti fisici per alcuni aspetti essenziali.

Modelli software che fanno parte del mondo reale che ci circonda, come servizi bancari o prenotazione di posti aerei. Il mondo intorno a noi cambia nel tempo, quindi anche il software corrispondente deve cambiare. Deve evolversi con la realtà che cambia. Gran parte di ciò che chiamiamo manutenzione del software riguarda in realtà la garanzia che il software tenga il passo con il mondo reale che viene modellato.

Otteniamo così un modello di processo in cui ripetiamo di volta in volta le fasi precedenti. Parliamo del ciclo di vita del software.

I metodi agili, il riutilizzo dei componenti, il software open source e la globalizzazione sono alcune delle tendenze relativamente recenti che hanno un enorme impatto sul modo in cui vediamo il campo. C'è un passaggio dalla produzione di software all'utilizzo del software, la cui principale conseguenza è che un'organizzazione di sviluppo perde il controllo su ciò che fornisce.

1.9 ULTERIORI LETTURE

Johnson (1998) descrive la storia iniziale dell'industria del software. Campbell-Kelly (2003) descrive la storia dell'industria fino al 2000 circa. Lo stato più recente della pratica è descritto in (Software, 2003). Finkelstein (2000) descrive lo stato dell'arte a partire dal 2000. Glass (2003) è una panoramica molto leggibile del campo dell'ingegneria del software, formulata come una raccolta di fatti e errori.

Per una discussione più elaborata delle differenze e delle somiglianze tra l'ingegneria del software e una disciplina ingegneristica matura, ad esempio la progettazione di ponti, si veda (Spector e Gifford, 1986). Leveson (1992) confronta l'ingegneria del software con lo sviluppo di motori a vapore ad alta pressione.

I quattro tipi di attività di manutenzione derivano da (Lientz e Swanson, 1980).

Il fallimento di Ariane è descritto in (Jézéquel e Meyer, 1997). Ho trovato il rapporto del team di indagine su www.cs.vu.nl/~hans/ariane5report.html. Un'elaborata discussione sugli incidenti di Therac-25 può essere trovata in (Leveson e Turner, 1993). L'indagine sul London Ambulance Service è descritta in (Page et al., 1993). Neumann (1995) è un libro interamente dedicato ai rischi legati al computer. Il bimestrale ACM Software Engineering Notes contiene una colonna "Rischi per il pubblico nei sistemi informatici", a cura di Peter Neumann, che riporta le grandi e piccole catastrofi causate dall'automazione. Flowers (1996) è una raccolta di storie sui sistemi informativi che hanno fallito, incluso il sistema LAS. (Kohno et al., 2004) e Raba (2004) discutono dei problemi con uno specifico sistema di voto elettronico. Ulteriori problemi dei sistemi di voto sono discussi in un numero speciale delle Comunicazioni dell'ACM (Neumann, 2004). Petroski (1994) è un libro meraviglioso sui fallimenti in ingegneria. (Avison et al., 2006) discutono diversi progetti che sono stati catastrofici, o quasi, a causa del compiacimento della gestione. Fairley e Wilshire (2003) raccontano la famosa storia della Vasa, una nave che affondò subito dopo il suo varo, ai fallimenti del progetto software. (Software, 1999) è un numero speciale con storie di progetti IT di successo.

Il codice etico di ingegneria del software ACM/IEEE è discusso in (Gotterbarn, 1999). Il testo del codice si trova anche in (Gotterbarn et al., 1999). Epstein (1997) è una raccolta di storie (di fantasia) che affrontano l'interazione tra etica e ingegneria del software. Oz (1994) discute le questioni etiche di un progetto di vita reale.

Esercizi

1. Definire il termine 'ingegneria del software'.

2.Quali sono le caratteristiche essenziali dell'ingegneria del software?

3. Quali sono le fasi principali di un progetto di sviluppo software?

4. Qual è la differenza tra verifica e convalida?

5. Definire i quattro tipi di attività di manutenzione.

6. Perché la documentazione di un progetto software è importante?

7. Spiegare la regola pratica 40–20–40 nell'ingegneria del software.

8. Qual è la differenza tra lo sviluppo del software e la manutenzione del software?

9. ♡Pensi che il modello lineare di sviluppo del software sia appropriato? In quali casi ritieni più appropriato un approccio agile? Potresti voler riconsiderare questo problema dopo aver letto il resto di questo libro.

10. ♡Discutere le principali differenze tra l'ingegneria del software e alcune altre discipline ingegneristiche, come la progettazione di ponti o la costruzione di case. Considereresti l'ingegneria del software all'avanguardia come una vera disciplina ingegneristica?

11. ♠ La qualità e la produttività sono questioni importanti nell'ingegneria del software. Si sostiene spesso che gli strumenti automatizzati (strumenti CASE) miglioreranno notevolmente sia la qualità che la produttività. Studia uno strumento CASE commerciale e valuta in che misura migliora il processo di sviluppo del software e i suoi risultati.

12. ♡ I medici hanno un giuramento di Ippocrate. Un simile impegno etico da parte degli ingegneri del software potrebbe essere determinante per aumentare la qualità dei sistemi software?

13. ♠ Si supponga di essere coinvolti in un progetto di automazione dell'ufficio nel settore della stampa. Il sistema da sviluppare ha lo scopo di supportare il lavoro degli editori di riviste. L'obiettivo di gestione di questo progetto è di risparmiare sui costi di manodopera; l'obiettivo dei redattori è quello di migliorare la qualità del loro lavoro. Discutere le possibili ramificazioni di questi obiettivi opposti sul progetto. Puoi tornare su questa domanda dopo aver letto **il capitolo 9** o (Hirschheim e Klein, **1989**).

14. ♡ Discutere la differenza tra lo sviluppo di software basato sui requisiti e lo sviluppo di software guidato dal mercato (Sawyer, **2001**).

15. ♡ Discutere l'impatto della globalizzazione sullo sviluppo del software (cfr. anche **il capitolo 20**).

16. ♡ Discutere come il software open source ha cambiato il panorama dello sviluppo del software (ad esempio, vedere (Riehle, **2007**)).

17. ♠ Studia sia la documentazione tecnica che quella per l'utente di un sistema a tua disposizione. Sei soddisfatto di loro? Discutere le loro possibili carenze e fornire rimedi per migliorarne la qualità.

18. ♠ Prendi un software che hai scritto più di un anno fa. È adeguatamente documentato? Ha un manuale utente? La logica progettuale si riflette nella documentazione tecnica? Puoi costruire una comprensione del sistema dalla sua documentazione che sia sufficiente per apportare modifiche non banali ad esso? Ripeti queste domande per un sistema scritto da uno dei tuoi colleghi.

19. ♠ Prova a raccogliere dati quantitativi dalla tua organizzazione che rivelino quanto sforzo viene speso per vari tipi di attività di manutenzione. Questi dati sono disponibili? In tal caso, il modello è come quello abbozzato nella **Sezione 1.3** ? In caso negativo, puoi spiegare le differenze?

20. ♠ Un'indagine della Computer Society del 1999 elenca i seguenti principi fondamentali candidati dell'ingegneria del software:

a) Applicare e utilizzare misurazioni quantitative nel processo decisionale.
b) Costruire con e per il riutilizzo.
c) Controllare la complessità con più prospettive e più livelli di astrazione.
d) Definire rigorosamente gli artefatti software.
e) Stabilire un processo software che offra flessibilità.
f) Attuare un approccio disciplinato e migliorarlo continuamente.
g) Investire nella comprensione del problema.
h) Gestire la qualità durante tutto il ciclo di vita nel modo più formale possibile.
i) Ridurre al minimo l'interazione dei componenti software.
j) Produrre software in modo graduale.
k) Stabilire obiettivi di qualità per ogni prodotto consegnabile.
l) Poiché il cambiamento è inerente al software, pianificarlo e gestirlo.
m) Poiché i compromessi sono inerenti all'ingegneria del software, rendili espliciti e documentali.
n) Per migliorare il design, studiare soluzioni precedenti a problemi simili.
o) L'incertezza è inevitabile nell'ingegneria del software. Identificalo e gestiscilo.

Per ciascuno di questi principi, indica se sei (fortemente) d'accordo o (fortemente) in disaccordo e perché. Potresti voler rivalutare questi principi dopo aver studiato il resto di questo libro.

Parte I Gestione del software

CONTENUTI

capitolo 2	Introduzione alla gestione dell'ingegneria del software	54
capitolo 3	Il ciclo di vita del software rivisitato	64
capitolo 4	Gestione della configurazione	85
Capitolo 5	Gestione delle persone e organizzazione del team	97
Capitolo 6	Sulla gestione della qualità del software	115
Capitolo 7	Stima dei costi	153
Capitolo 8	Pianificazione e controllo del progetto	189

I progetti di sviluppo software spesso coinvolgono più persone per un periodo di tempo prolungato. I grandi progetti possono durare anche diversi anni e coinvolgere centinaia di persone. Tali progetti devono essere attentamente pianificati e controllati. Gli aspetti principali che meritano la continua attenzione dei project manager sono introdotti nel Capitolo 2 e ulteriormente trattati nei Capitoli 3 – 7 : progresso, informazioni, persone, qualità, costi e tempi. La parte gestionale si conclude con il Capitolo 8, in cui si riconciliano i vari approcci abbozzati nei Capitoli 3 – 7.

2 Introduzione alla gestione dell'ingegneria del software

OBIETTIVI FORMATIVI

- Essere a conoscenza dei contenuti di un piano di progetto
- Comprendere le dimensioni principali lungo le quali viene controllato un progetto di sviluppo software

I progetti di sviluppo software spesso coinvolgono più persone per un periodo di tempo prolungato. I grandi progetti possono durare anche diversi anni e coinvolgere centinaia di persone. Tali progetti devono essere attentamente **pianificati** e **controllati**. In questo capitolo vengono introdotti gli aspetti principali che meritano la continua attenzione dei project manager.

Non è facile portare a termine con successo un progetto di sviluppo software. Questo libro tratta principalmente gli aspetti tecnici dello sviluppo del software: progettazione, specifica, implementazione e test di sistemi software. Man mano che impariamo a controllare meglio questi aspetti, impareremo anche a soddisfare meglio le richieste dei nostri clienti. Tuttavia, gli aspetti organizzativi e gestionali dei progetti di sviluppo software sono importanti almeno quanto gli aspetti tecnici.

Prima di intraprendere una discussione su questi aspetti organizzativi e gestionali, prestiamo prima un po' di attenzione ai confini di un progetto di sviluppo software così come sono disegnati in questo libro.

Un progetto di sviluppo software di solito non viene avviato in completo isolamento. Ci sono altri progetti all'interno dell'organizzazione su cui questo particolare progetto deve essere sintonizzato, le priorità tra i progetti devono essere decise, ecc. Il termine " **pianificazione delle informazioni** " è spesso usato per riferirsi a questo processo di pianificazione del meta-progetto.

Anche in senso più tecnico, i progetti non vengono avviati in modo isolato. Per aumentare l'interoperabilità tra i sistemi, le linee guida generali riguardanti l'uso di determinati standard, i formati di scambio dei dati, le politiche di sicurezza, il layout delle pagine Web e così via sono stabilite per l'intera organizzazione e imposte a ogni progetto. Nello sviluppo della linea di prodotti, l'architettura della linea di prodotti guida lo sviluppo dei singoli prodotti.

Queste regole di superamento del progetto si traducono in una serie di condizioni al contorno per ciascun progetto, proprio come i regolamenti urbanistici stabiliscono le condizioni per un progetto di costruzione. Stabilire queste regole a livello aziendale è un problema di per sé e non verrà affrontato qui. (Presteremo, tuttavia, ampia attenzione ad alcuni problemi che generalmente superano i confini dei singoli progetti di sviluppo software, come il controllo della configurazione, la garanzia della qualità e lo sviluppo della linea di prodotti.)

In un senso più tecnico, anche il software non è generalmente sviluppato isolatamente. Nella maggior parte dei casi, il software non viene scritto da zero. Deve interfacciarsi con il software esistente, estendere il software esistente, utilizzare le librerie di subroutine esistenti, basarsi su un framework esistente e così via.

In un certo senso, la nozione di "progetto di sviluppo software" è impropria. Non sviluppiamo solo software, sviluppiamo sistemi. In generale, un sistema trasforma gli input in output. Il software è un ingrediente importante dei sistemi che sviluppiamo, ma non è affatto l'unico ingrediente. La documentazione tecnica e per l'utente, l'hardware, le procedure che regolano l'uso del sistema, e anche le persone che utilizzano il software, possono essere considerati come parte di quel sistema stesso.

Si consideri, ad esempio**, un sistema per l'automazione delle biblioteche**. Il sistema conterrà vari componenti software, come un componente database per memorizzare informazioni su libri e clienti e un componente di interazione per elaborare le richieste degli utenti. Oltre allo sviluppo di questi componenti, occorre prestare attenzione a questioni quali:

- tecniche per identificare i libri elettronicamente, come uno schema di codici a barre;
- la selezione e l'acquisizione di hardware speciale sia per produrre un codice identificativo per i nuovi libri che per scansionare il codice identificativo;
- istituire uno schema per fornire a tutti i libri un codice identificativo;
- istruzione dei dipendenti della biblioteca per gestire le nuove tipologie di apparecchiature (materiale e corsi di formazione, procedure operative e così via);
- produzione di documentazione di facile utilizzo per i clienti della biblioteca;
- Problemi di accessibilità al Web, ad esempio se il catalogo può essere sfogliato o se i libri possono essere prenotati online.

Ogni volta che la nozione di "progetto di sviluppo software" viene utilizzata nel testo seguente, dovrebbe essere intesa in questo senso più ampio. Ciò è illustrato graficamente nella Figura 2.1.

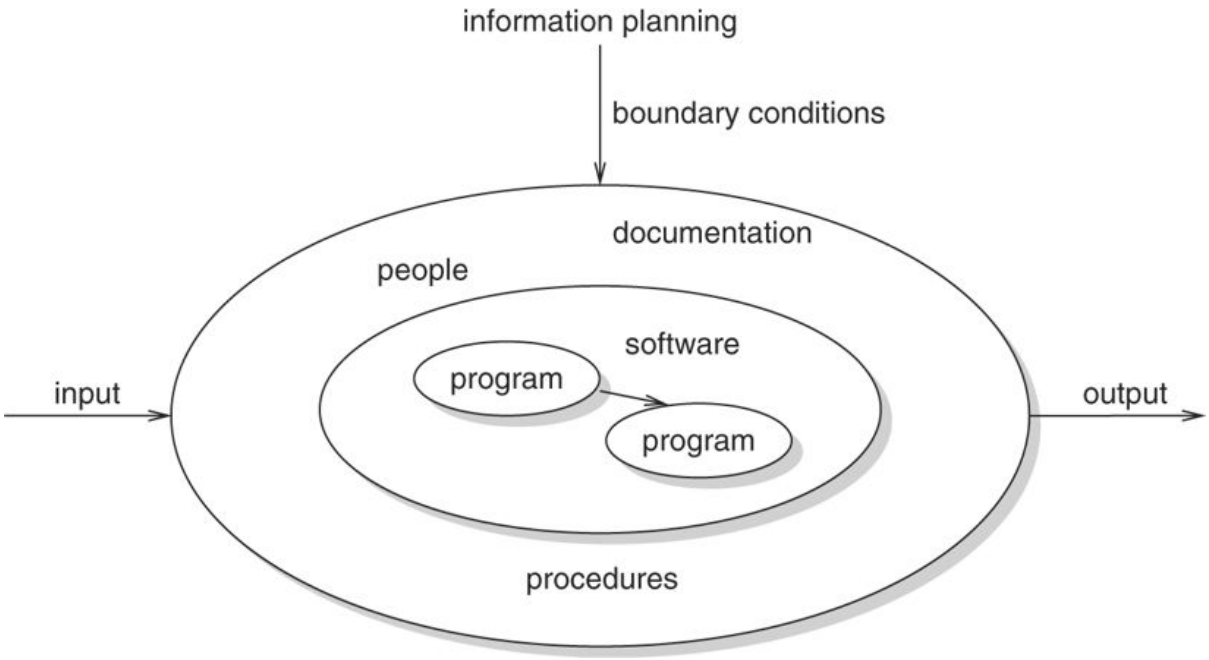


Figura 2.1 La vista dei sistemi di un progetto di sviluppo software

Pertanto, i nostri sistemi comprendono una serie di componenti. In senso stretto, il componente software stesso può anche essere costituito da una serie di componenti interagenti. Questi ultimi componenti corrispondono ai programmi come li conosciamo dai libri di testo introduttivi di informatica. In generale, un progetto di sviluppo software si traduce in un insieme di componenti che insieme ci forniscono la funzionalità desiderata.

Date le condizioni al contorno di un progetto, è possibile che venga avviato un progetto di sviluppo software. La pianificazione del progetto è il primo passo da intraprendere. Parte di questo processo di pianificazione consiste nell'identificare le caratteristiche del progetto e il loro impatto sul processo di sviluppo. Il risultato della fase di progettazione è racchiuso in un documento, il piano di **progetto,** che mira a fornire un quadro chiaro del progetto sia ai clienti che al team di sviluppo. I contenuti del piano di progetto sono discussi nella Sezione 2.1 .

Durante l'esecuzione del progetto, è necessario gestire una serie di elementi: tempo, informazioni, organizzazione, qualità e denaro (vedi Sezione 2.2). Ciascuno di questi elementi è ulteriormente approfondito in un capitolo separato.

2.1 PIANIFICARE UN PROGETTO DI SVILUPPO SOFTWARE

Prima di intraprendere un progetto di sviluppo software, è necessario pianificarlo attentamente. La pianificazione comporta, tra l'altro, una valutazione delle proprietà del progetto che possono influenzare il processo di sviluppo. Alcune proprietà, tuttavia, non saranno sufficientemente comprese fino al termine della fase di ingegneria dei requisiti. Come per molti altri aspetti di un progetto di sviluppo software, la pianificazione non è un'attività

unica. Piuttosto, è di natura altamente dinamica. Il piano di progetto può fungere da guida durante il progetto.

La quantità di pianificazione anticipata dipende dalle caratteristiche del problema in questione. Nei progetti altamente esplorativi, in cui i requisiti sono in gran parte sconosciuti all'inizio, una pianificazione precoce troppo rigorosa può essere soffocante e può aumentare la probabilità di gravi fallimenti. Un atteggiamento rigoroso "pianifica il lavoro e lavora secondo il piano" non funziona in queste circostanze. Piuttosto, tali progetti richiedono una pianificazione iniziale nominale e uno stile di gestione che incoraggi a rispondere al cambiamento. Ciò si riflette nel contenuto e nelle dimensioni del piano di progetto. **I capitoli 3** e **8** discutono ulteriormente le differenze tra i cosiddetti approcci agili e basati sulla pianificazione allo sviluppo del software.

I principali componenti di un piano di progetto (**project plan**) sono:

1. **Introduzione** Nell'introduzione al piano di progetto, vengono forniti il background e la storia del progetto, insieme ai suoi obiettivi, i risultati del progetto, i nomi delle persone responsabili e una sintesi del progetto.
2. **Modello di processo** Nel **Capitolo 1** è stato introdotto un semplice modello del ciclo di vita per discutere le varie attività da affrontare in un progetto di sviluppo software. Esistono molte varianti di questo modello di processo, alcune delle quali sono discusse nel **Capitolo 3**. Per ogni progetto si deve decidere l'esatto modello di processo da seguire: quali attività intraprendere, quali pietre miliari possono essere identificate, come si accerta se tali pietre miliari vengono raggiunte e quali sono i percorsi critici. Diversi tipi di progetti hanno caratteristiche diverse e quindi richiedono modelli di processo diversi.
   Ad esempio, il project plan potrebbe stabilire che si utilizzerà un processo agile SCRUM.
3. **Organizzazione del progetto** Il rapporto del progetto con altri enti e l'organizzazione del progetto stesso sono trattati in questa rubrica. Il progetto avrà una relazione con l'organizzazione utente, l'organizzazione madre ed eventualmente con altre organizzazioni. I potenziali utenti saranno, di volta in volta, coinvolti nel progetto. Il piano del progetto deve indicare quali informazioni, servizi, risorse e strutture devono essere fornite dagli utenti e quando devono essere forniti.
   All'interno del team di progetto possono essere identificati vari ruoli: project manager, tester, programmatore, analista, ecc. È necessario delineare chiaramente questi ruoli e identificare le responsabilità di ciascuno di essi. Se ci sono lacune nelle conoscenze richieste per ricoprire uno di questi ruoli, è necessario identificare la formazione e l'istruzione necessarie per colmare queste lacune. Diverse forme di organizzazione del team sono discusse nel **Capitolo 5** .
4. **Standard, linee guida, procedure** I progetti software sono grandi progetti. Di solito, molte persone sono coinvolte. Occorre quindi una forte disciplina di lavoro, in cui ogni persona coinvolta segua gli standard, le linee guida e le procedure concordate. Oltre ad essere dichiarati sulla carta, molti di questi possono essere supportati o applicati da strumenti. Di estrema importanza sono gli accordi chiari sulla documentazione: quando consegnare la

documentazione, come valutare la qualità della documentazione, come garantire l'aggiornamento della documentazione? In larga misura, questi standard e procedure saranno descritti in documenti separati, come il Piano di controllo della configurazione o il Piano di garanzia della qualità.

Per esempio un progetto potrebbe dichiarare di seguire le convenzioni per la programmazione Java come definite da Oracle[1].

5. **Attività di gestione** sono guidate da obiettivi e priorità fissati per il progetto. Ad esempio, la direzione dovrà presentare relazioni periodiche sullo stato e sullo stato di avanzamento del progetto. Dovrà anche seguire alcune priorità nel bilanciamento di requisiti, tempi e costi.
6. **Rischi** I potenziali rischi devono essere identificati il prima possibile. Ci saranno sempre dei rischi: l'hardware potrebbe non essere consegnato in tempo, il personale qualificato potrebbe non essere disponibile quando necessario, le informazioni critiche potrebbero mancare quando è necessario e così via. È piuttosto ingenuo supporre che un progetto di sviluppo software funzioni sempre senza intoppi. Anche in campi consolidati, come quello delle costruzioni, c'è sempre qualcosa che non va. Si dovrebbe diagnosticare precocemente i rischi di un progetto software e fornire misure per affrontarli; vedere anche **il capitolo 8**. Più incerti sono i vari aspetti del progetto, maggiori sono i rischi.
7. **Personale** In momenti diversi, il progetto richiederà diverse quantità di personale, con competenze diverse. L'inizio, la durata, l'importo e la competenza delle categorie di personale sono elencati in questa voce.
8. **Metodi e tecniche** In questa sezione vengono forniti i metodi e le tecniche da utilizzare durante l'ingegneria dei requisiti, la progettazione, l'implementazione e le prove. In genere, qui viene descritto anche il modo in cui viene gestito il controllo della versione e della configurazione per i componenti software. Gran parte della documentazione tecnica sarà prodotta durante queste fasi. Si deve quindi precisare come sarà curata questa documentazione.

   Viene descritto l'ambiente di prova e le apparecchiature di prova necessari. Durante il test, viene normalmente esercitata una pressione considerevole sull'apparecchiatura di test. Pertanto, questa attività deve essere pianificata con attenzione. L'ordine in cui i componenti vengono integrati e testati deve essere indicato esplicitamente. Inoltre, devono essere indicate le procedure da seguire durante i test di accettazione, ovvero i test sotto la supervisione dell'utente. I test sono discussi nel **Capitolo 13**.
9. **Garanzia di qualità** Quale organizzazione e procedure verranno utilizzate per garantire che il software in fase di sviluppo soddisfi i requisiti di qualità dichiarati? I molti aspetti di un Piano di Assicurazione della Qualità possono anche essere trattati in un documento separato. Il tema della garanzia della qualità è discusso nel **Capitolo 6** .

[1] https://www.oracle.com/java/technologies/javase/codeconventions-contents.html

10. **Pacchetti di lavoro (workpackages)** I progetti più grandi devono essere suddivisi in attività, parti gestibili che possono essere allocate ai singoli membri del team. Ciascuna di queste attività deve essere identificata nel piano di progetto. La scomposizione gerarchica del progetto è rappresentata in una struttura di scomposizione del lavoro (vedi anche **Sezione 8.4**).
11. **Risorse** Durante il progetto sono necessarie molte risorse. L'hardware, i cicli della CPU e gli strumenti necessari per supportare il progetto sono elencati in questa voce. Occorre inoltre indicare il personale necessario per le varie fasi del processo.
12. **Budget** e programma Il budget totale per il progetto deve essere assegnato alle varie attività come indicato nella struttura di ripartizione del lavoro. Anche le attività devono essere programmate in tempo, ad esempio utilizzando un grafico PERT (vedi **Sezione 8.4**). In questa rubrica è indicato anche il modo in cui vengono tracciate le risorse e le altre spese. Il tema della stima dei costi e dei tempi è trattato ampiamente nel **Capitolo 7**.
13. **Cambiamenti** È stato affermato in precedenza che i cambiamenti sono inevitabili. Bisogna garantire che questi cambiamenti siano gestiti in modo ordinato. Sono quindi necessarie procedure chiare su come verranno gestite le modifiche proposte. Se il processo è agile, ogni iterazione comporta modifiche e queste vengono gestite in modo leggero. In realtà, non sono visti come cambiamenti. Nei processi più pesanti, ogni modifica proposta deve essere registrata e rivista. Quando una richiesta di modifica è stata approvata, è necessario stimarne l'impatto (costo). Infine, la modifica deve essere incorporata nel progetto. Le modifiche che vengono immesse tramite la porta sul retro portano a codice strutturato male, documentazione inadeguata e superamento di costi e tempi. Poiché le modifiche portano a versioni diverse sia della documentazione che del codice, le procedure da seguire per gestire tali modifiche vengono spesso gestite nel contesto di un piano di controllo della configurazione.
14. **Consegna** Devono essere indicate le procedure da seguire per la consegna dell'impianto al cliente.

Il piano del progetto mira a fornire un quadro chiaro del progetto sia al cliente che al team di progetto. Se gli obiettivi non sono chiari, non saranno raggiunti.

Nonostante un'attenta pianificazione, le sorprese continueranno a sorgere durante il progetto. Tuttavia, un'attenta pianificazione all'inizio porta a meno sorprese e rende il progetto meno vulnerabile a queste sorprese. Il piano del progetto affronta una serie di domande che anticipano possibili eventi futuri. Fornisce procedure ordinate per affrontare tali eventi, in modo che possano essere raggiunte decisioni giustificate.

2.2 CONTROLLO DI UN PROGETTO DI SVILUPPO SOFTWARE

Dopo che un piano di progetto è stato redatto e approvato, l'esecuzione del progetto può iniziare. Durante il progetto, il controllo deve essere esercitato lungo le seguenti dimensioni:

- tempo,
- informazione,

- organizzazione,
- qualità,
- i soldi.

L'avanzamento di un progetto di sviluppo software (L'aspetto **temporale)** è difficile da misurare. Prima che il sistema proposto sia terminato, c'è solo una (grande) pila di carta. Espressioni come "il 90% del codice è stato scritto" dovrebbero essere prese con le pinze. Di solito viene fornita un'immagine troppo rosea dello stato attuale delle cose. L'approccio graduale introdotto nel **Capitolo 1**, e le sue varianti, mirano a fornire al gestore uno strumento per misurare e controllare i progressi. Il tempo necessario per realizzare un impianto è ovviamente correlato alla dimensione dell'impianto, e quindi alla manodopera totale richiesta. Sistemi più grandi richiedono più tempo per essere sviluppati, anche se potremmo cercare di ridurre i tempi di sviluppo assegnando più personale. Parte del problema del controllo per i progetti di sviluppo software è quello di scambiare tempo con le persone. L'aggiunta di più persone per ridurre i tempi di sviluppo non è gratuita. Più persone sono coinvolte, più tempo sarà necessario per il coordinamento e la comunicazione. Dopo un certo punto, l'aggiunta di più persone allunga effettivamente i tempi di sviluppo. Parte del problema del controllo del tempo è formulato nella legge di Brooks: "Aggiungere persone a un progetto in ritardo lo fa solo dopo". Torneremo su questo problema nel **Capitolo 7**.

L'i**nformazione** che deve essere gestita, soprattutto, è la **documentazione**. Oltre alla documentazione tecnica e per l'utente, ciò comporta anche la documentazione sul progetto stesso. La documentazione relativa al progetto include elementi quali: lo stato attuale delle cose, le modifiche concordate e le decisioni prese. Questo tipo di documentazione può essere gestita al meglio nel contesto della gestione della configurazione. Nei progetti agili, durante lo sviluppo viene prestata meno attenzione alla documentazione. La conoscenza necessaria è tacita; risiede nella testa delle persone coinvolte. Ma anche qui, una volta che il sistema è pronto e consegnato ai propri clienti, è necessario fornire la documentazione.

Tutti i membri del team di sviluppo devono comprendere il loro ruolo nel team e cosa ci si aspetta da loro. È molto importante che queste aspettative siano chiare a tutte le persone coinvolte. Le aspettative non dette e poco chiare portano a situazioni in cui i singoli membri del team stabiliscono i propri obiettivi, consciamente o inconsciamente. Questi aspetti **organizzativi meritano la continua attenzione del project manager.** L'organizzazione di un team e il coordinamento delle persone coinvolte dipenderanno, almeno in parte, dalle caratteristiche del progetto e del suo ambiente. Questa dipendenza deve essere riconosciuta e presa in considerazione durante la creazione di un team di progetto.

L'aspetto della **qualità** è di fondamentale importanza. I clienti non sono soddisfatti delle soluzioni puramente tecniche offerte da specialisti informatici. Vogliono sistemi che soddisfino le loro reali esigenze. I requisiti di qualità per il software e per il suo processo di sviluppo sono spesso in conflitto tra loro. Al momento dell'architettura, i requisiti di qualità sono bilanciati in un dialogo con tutte le parti interessate coinvolte. Durante un progetto,

dobbiamo valutare se i requisiti di qualità sono soddisfatti o meno. Questa valutazione della qualità deve avvenire su base regolare, in modo che possano essere intraprese azioni tempestive. La qualità non è una funzionalità aggiuntiva, deve essere integrata.

Controllare le spese (L'aspetto del **denaro)** significa in gran parte controllare i costi del lavoro. Sebbene il costo dell'hardware e degli strumenti non possa essere ignorato, di solito è possibile stimarlo in modo abbastanza preciso all'inizio del progetto. Inoltre, questi sono solitamente molto meno problematici dei costi del personale.

Stimare il costo del software significa che dobbiamo stimare la manodopera necessaria per costruire il software. La manodopera necessaria dipende molto dalle dimensioni del software, ad esempio misurata come la quantità di codice da fornire. Molti altri fattori, però, influenzano questo costo o, in alternativa, la produttività con cui il software può essere prodotto. Un team ben bilanciato con persone esperte sarà molto più produttivo di un team appena formato con persone inesperte. Anche vincoli di qualità estremamente severi, come un'affidabilità molto elevata o un tempo di risposta molto rapido, possono ridurre notevolmente la produttività.

Sono stati proposti numerosi modelli che tentano di quantificare l'effetto di questi diversi fattori di costo sulla manodopera richiesta (cfr. **capitolo 7**). Invece di stimare prima la dimensione e poi il suo costo, si può anche impostare prima una soglia di costo e lavorare in modo incrementale, prima sui requisiti più urgenti dell'utente e, se il tempo lo consente, su quelli meno urgenti. Oppure si può concordare una prima soglia e decidere se verranno spesi più soldi al raggiungimento di tale soglia. Questi approcci incrementali alla stima dei costi si adattano bene allo sviluppo agile del progetto.

Lo sviluppo del software è un processo molto laborioso. Una delle nostre speranze è che strumenti migliori e un maggiore utilizzo di tali strumenti portino a un aumento significativo della produttività e, di conseguenza, a una diminuzione significativa dei costi coinvolti nello sviluppo del software. Un secondo modo per aumentare notevolmente la produttività consiste nell'utilizzare il software anziché crearlo da soli. Entrambi questi argomenti sono discussi nei capitoli successivi. Man mano che queste tendenze continuano, lo sviluppo del software inizia a diventare un'attività ad alta intensità di capitale, piuttosto che ad alta intensità di manodopera (Wegner, **1984**).

La valutazione continua del progetto rispetto a questi aspetti di controllo è della massima importanza e, di volta in volta, porterà ad adeguamenti di tempo, costi, organizzazione, informazioni o qualità, o una loro combinazione. La gestione dei progetti è un'attività molto dinamica.

Per poter controllare adeguatamente un progetto, abbiamo bisogno di dati quantitativi, che vengono raccolti durante l'esecuzione del progetto. Ad esempio, i dati sugli errori scoperti durante i test unitari possono aiutarci a stimare l'ulteriore sforzo necessario per i test. I dati

sul tempo e sull'impegno spesi fino a un punto specifico ci guideranno nella ristima del programma e del costo. Misurare è conoscere.

Questi dati sono preziosi anche in una valutazione post mortem del progetto. In una valutazione post mortem, valutiamo il progetto attuale al fine di migliorare le nostre prestazioni su progetti ancora a venire: cosa abbiamo sbagliato, cosa abbiamo imparato, cosa è necessario fare di diverso nel prossimo progetto?

Sfortunatamente, in pratica vengono raccolti pochissimi dati concreti, per non parlare di conservati per un uso successivo. La maggior parte delle organizzazioni di sviluppo software ha poche informazioni su ciò che stanno facendo. Tendono ad operare in modo un po' caotico, soprattutto di fronte a una crisi. Identificando i fattori chiave che influenzano la controllabilità del processo di sviluppo del software, potremmo trovare modi per migliorarlo. Questo argomento viene ulteriormente trattato nella **Sezione 6.6**, dove si discute il Capability Maturity Model.

2.3 RIEPILOGO

Questo capitolo fornisce un'introduzione alla gestione dei progetti di ingegneria del software.

Prima di intraprendere un progetto di sviluppo software, è necessario pianificarlo attentamente. Questo processo di pianificazione si traduce in un documento, il piano di progetto, che fornisce un quadro chiaro del progetto sia ai clienti che al team di progetto.

Una volta redatto il piano di progetto e avviato il progetto, è necessario controllarne l'esecuzione. Abbiamo identificato cinque entità che richiedono la nostra continua attenzione per il controllo del progetto:

- **Tempo**: come valutiamo i progressi verso gli obiettivi del progetto? Di solito, viene seguito un approccio graduale che mira a fornire alla direzione un modo per misurare e controllare i progressi.
- **Informazioni**: come gestiamo i documenti che vengono prodotti nel corso di un progetto? Nello sviluppo basato sulla pianificazione, il mantenimento dell'integrità della serie di documenti e la gestione di tutte le richieste di modifica richiedono procedure attente.
- **Organizzazione**: come organizziamo il team di progetto e coordiniamo le attività dei membri del team?
- **Qualità**: come definiamo e valutiamo i requisiti di qualità sia per il processo di sviluppo che per il prodotto risultante?
- **Denaro**: come si stima il costo di un progetto? Questi costi sono in larga misura determinati dalle dimensioni del software.

Ciascuno di questi aspetti di controllo è ulteriormente elaborato in un capitolo separato (**Capitoli 3 – 7**). Le varie dimensioni del controllo del progetto sono poi riconciliate nel **Capitolo 8** .

Esercizi

1. In che senso l'espressione "progetto di sviluppo software" è un termine improprio?

2. Quali sono i principali componenti di un piano di progetto?

3. Elencare cinque dimensioni lungo le quali deve essere controllato un progetto di sviluppo software.

4. In che modo lo sviluppo del software può diventare un'attività ad alta intensità di capitale, piuttosto che ad alta intensità di manodopera?

5. ♠ Considera un progetto di sviluppo software in cui sei stato coinvolto. Il progetto aveva un piano di progetto? Il piano del progetto ha affrontato le questioni elencate nella **Sezione 2.1**? Se alcuni di questi problemi non fossero stati affrontati, pensi che avrebbe aiutato il progetto se lo fossero stati?

6. ♡ Pensi che i dati quantitativi del progetto siano importanti? In che modo può contribuire alla pianificazione del progetto?

7. ♡ In che modo un piano di progetto per un progetto agile differirebbe da quello per un progetto guidato dalla pianificazione?

8. ♠ Considera ancora una volta un progetto di sviluppo software in cui sei stato coinvolto. In che misura le questioni ambientali, come la formazione degli utenti e le procedure di lavoro, sono state adeguatamente affrontate nel progetto?

9. ♡ Un programma scritto per uso personale impone requisiti piuttosto meno severi rispetto a un prodotto che deve essere utilizzato anche da altre persone. Secondo (Brooks, **1995**), quest'ultimo può richiedere uno sforzo tre volte maggiore. Discutere le possibili ragioni di questo notevole aumento dei costi.